

**PENERAPAN ARSITEKTUR *CLIENT DATA LAYER* PADA  
DASHBOARD ANALISIS SENTIMEN MENGGUNAKAN  
TANSTACK QUERY DENGAN METODE FOUNTAIN**

**PROPOSAL TUGAS AKHIR**

Diajukan Sebagai Syarat Untuk Pengajuan Tugas Akhir Pada Program Studi Sarjana Terapan  
Teknologi Rekayasa Perangkat Lunak Jurusan Bisnis dan Informatika Politeknik Negeri  
Banyuwangi



**Oleh:**

**NASRUL FAHMI ULUMUDDIN**

**NIM 362258302204**

**PROGRAM STUDI SARJANA TERAPAN TEKNOLOGI REKAYASA  
PERANGKAT LUNAK  
JURUSAN BISNIS DAN INFORMATIKA  
POLITEKNIK NEGERI BANYUWANGI  
2026**

**PENERAPAN ARSITEKTUR *CLIENT DATA LAYER* PADA  
DASHBOARD ANALISIS SENTIMEN MENGGUNAKAN  
TANSTACK QUERY DENGAN METODE FOUNTAIN**

**PROPOSAL TUGAS AKHIR**

Diajukan kepada Jurusan Bisnis dan Informatika Politeknik Negeri Banyuwangi Sebagai Syarat  
Untuk Pengajuan Tugas Akhir Pada Program Studi Sarjana Terapan Teknologi Rekayasa  
Perangkat Lunak

Oleh:

**NASRUL FAHMI ULUMUDDIN**  
362258302204

Pembimbing:

Sepyan Purnama Kristanto, S.Kom., M.Kom  
Arum Andary Ratri, S.Si., M.Si.

**PROGRAM STUDI SARJANA TERAPAN TEKNOLOGI REKAYASA  
PERANGKAT LUNAK  
JURUSAN BISNIS DAN INFORMATIKA  
POLITEKNIK NEGERI BANYUWANGI**

**2026**

# **LEMBAR PERSETUJUAN**

Proposal Tugas Akhir dengan Judul

## **PENERAPAN ARSITEKTUR *CLIENT DATA LAYER* PADA DASHBOARD ANALISIS SENTIMEN MENGGUNAKAN TANSTACK QUERY DENGAN METODE FOUNTAIN**

Disusun oleh:

**NASRUL FAHMI ULUMUDDIN**

**NIM 362258302204**

telah memenuhi syarat dan disetujui oleh Dosen Pembimbing untuk dilaksanakan Seminar  
Proposal Tugas Akhir bagi yang bersangkutan.

Banyuwangi, 11 Februari 2026

Mengetahui,  
Koordinator Program Studi,

Disetujui,  
Dosen Pembimbing 1,

Lutfi Hakim, S.Pd., M.T.  
NIP. 199203302019031012

Sepyan Purnama Kristanto, S.Kom., M.Kom  
NIP. 199009052019031024

Banyuwangi, 11 Februari 2026

Disetujui,  
Dosen Pembimbing 2,

Arum Andary Ratri, S.Si., M.Si.  
NIP. 199209212020122021

# **Penerapan Arsitektur *Client Data Layer* pada Dashboard Analisis Sentimen Menggunakan TanStack Query dengan Metode Fountain**

Oleh  
NASRUL FAHMI ULUMUDDIN  
NIM: 362258302204

## **ABSTRAK**

Abstrak adalah sebuah ringkasan singkat yang menjelaskan secara umum tentang isi dari laporan tugas akhir. Abstrak ditulis dalam tiga (3) paragraf yang berisi beberapa kalimat yang menyatakan tujuan, metode, hasil, dan kesimpulan dari laporan tugas akhir. Paragraf pertama berisi latar belakang dan tujuan tugas akhir. Paragraf kedua berisi metode dan pembahasannya. Paragraf ketiga berisi hasil dan simpulan dari tugas akhir yang dikerjakan.

Abstrak harus menjelaskan secara jelas dan singkat apa yang dibahas dalam laporan tugas akhir, mengapa penelitian ini penting dan apa yang ditemukan dari penelitian tersebut. Abstrak harus ditulis dengan bahasa yang mudah dipahami dan harus mencakup informasi penting yang dibahas dalam laporan tugas akhir.

Abstrak harus mengandung kata-kata yang relevan dengan laporan tugas akhir dan ditulis dengan bahasa yang formal dan akademik. Abstrak merupakan bagian penting dari sebuah laporan tugas akhir karena merupakan bagian yang pertama kali dibaca oleh pembaca dan harus dapat memberikan gambaran yang jelas tentang isi dari laporan tugas akhir. Oleh karena itu, abstrak harus ditulis dengan baik dan sebaik mungkin agar dapat memberikan gambaran yang jelas tentang laporan tugas akhir yang ditulis. Panjang abstrak sebaiknya dicukupkan dalam satu halaman, termasuk kata kunci. Tiga kata kunci dipandang cukup, yang masing-masingnya memuat paduan kata utama, yang dapat merepresentasikan isi Abstrak.

Kata kunci: Konsep Abstrak, Komponen Abstrak, Kata Kunci.

# **Implementation of Client Data Layer Architecture Using TanStack Query for a Sentiment Analysis Dashboard**

by:

NASRUL FAHMI ULUMUDDIN

NIM: 362258302204

## **ABSTRACT**

The abstract is a short summary that explains in general the contents of the final assignment report. The abstract is written in three (3) paragraphs containing several sentences stating the objectives, methods, results and conclusions of the final assignment report. The first paragraph contains the background and objectives of the final assignment. The second paragraph contains the method and discussion. The third paragraph contains the results and conclusions of the final assignment carried out.

The abstract must explain clearly and concisely what is discussed in the final project report, why this research is important and what was found from the research. The abstract must be written in language that is easy to understand and must include important information discussed in the final project report.

The abstract must contain words that are relevant to the final project report and be written in formal and academic language. The abstract is an important part of a final assignment report because it is the part that is first read by the reader and must be able to provide a clear picture of the contents of the final assignment report. Therefore, the abstract must be written well and as well as possible in order to provide a clear picture of the final project report being written. The length of the abstract should be limited to one page, including keywords. Three keywords are considered sufficient, each of which contains a combination of main words, which can represent the contents of the Abstract.

Key words: Abstract Concepts, Abstract Components, Key Words.

## DAFTAR ISI

|                                                    |             |
|----------------------------------------------------|-------------|
| <b>HALAMAN SAMPUL.....</b>                         | <b>i</b>    |
| <b>LEMBAR PERSETUJUAN PROPOSAL .....</b>           | <b>ii</b>   |
| <b>ABSTRAK .....</b>                               | <b>iii</b>  |
| <b>ABSTRACT.....</b>                               | <b>iv</b>   |
| <b>DAFTAR ISI .....</b>                            | <b>v</b>    |
| <b>DAFTAR SINGKATAN .....</b>                      | <b>vii</b>  |
| <b>DAFTAR GAMBAR .....</b>                         | <b>viii</b> |
| <b>DAFTAR TABEL .....</b>                          | <b>ix</b>   |
| <b>BAB 1 PENDAHULUAN .....</b>                     | <b>1</b>    |
| 1.1 Latar Belakang .....                           | 1           |
| 1.2 Rumusan Masalah .....                          | 2           |
| 1.3 Tujuan .....                                   | 3           |
| 1.4 Manfaat.....                                   | 3           |
| 1.5 Batasan Masalah .....                          | 3           |
| <b>BAB 2 TINJAUAN PUSTAKA .....</b>                | <b>5</b>    |
| 2.1 Landasan Teori.....                            | 5           |
| 2.1.1 UMKM dan Digitalisasi .....                  | 5           |
| 2.1.2 Media Sosial sebagai Sumber Data .....       | 6           |
| 2.1.3 Analisis Sentimen pada Media Sosial .....    | 6           |
| 2.1.4 Dashboard Analitik dan Visualisasi Data..... | 7           |
| 2.1.5 <i>Blackbox Testing</i> .....                | 9           |
| 2.1.6 <i>Arsitektur Client Data Layer</i> .....    | 9           |
| 2.1.7 <i>REST API</i> .....                        | 15          |
| 2.1.8 React .....                                  | 16          |
| 2.1.9 TanStack Query (React Query) .....           | 17          |
| 2.1.10 Metode Fountain.....                        | 18          |
| 2.2 Penelitian Terkait .....                       | 21          |
| <b>BAB 3 METODOLOGI PENELITIAN.....</b>            | <b>24</b>   |
| 3.1 Waktu dan Jadwal penelitian .....              | 24          |
| 3.1.1 Waktu Pelaksanaan Penelitian .....           | 24          |
| 3.1.2 Jadwal Kegiatan Penelitian.....              | 24          |
| 3.2 Metode Fountain.....                           | 26          |
| 3.2.1 Analysis .....                               | 27          |
| 3.2.2 Requirement Specification .....              | 28          |

|                             |                               |           |
|-----------------------------|-------------------------------|-----------|
| 3.2.3                       | <i>Design</i> .....           | 29        |
| 3.2.4                       | Coding (Implementation) ..... | 41        |
| 3.2.5                       | Testing .....                 | 42        |
| <b>DAFTAR PUSTAKA</b> ..... |                               | <b>55</b> |

## DAFTAR SINGKATAN

|      |   |                                |
|------|---|--------------------------------|
| FWHM | : | <i>Full width half maximum</i> |
| rms  | : | <i>root mean square</i>        |
| RFS  | : | <i>Rotary forcespinning</i>    |
| PVP  | : | Polivinil pirolidon            |
| SI   | : | Satuan Internasional           |
| SI   | : | Satuan Internasional           |
| SI   | : | Satuan Internasional           |
| SI   | : | Satuan Internasional           |
| SI   | : | Satuan Internasional           |
| SI   | : | Satuan Internasional           |
| SI   | : | Satuan Internasional           |
| SI   | : | Satuan Internasional           |
| SI   | : | Satuan Internasional           |
| SI   | : | Satuan Internasional           |



## DAFTAR GAMBAR

|                    |                                           |    |
|--------------------|-------------------------------------------|----|
| <b>Gambar 2.1</b>  | Arsitektur <i>Client Data Layer</i> ..... | 10 |
| <b>Gambar 2.2</b>  | Fountain SDLC Model .....                 | 19 |
| <b>Gambar 3.1</b>  | Use Case Diagram .....                    | 29 |
| <b>Gambar 3.2</b>  | Entity Relationship Diagram .....         | 31 |
| <b>Gambar 3.3</b>  | Alur Sistem Keseluruhan .....             | 32 |
| <b>Gambar 3.4</b>  | Diagram Arsitektur <i>Frontend</i> .....  | 34 |
| <b>Gambar 3.5</b>  | Cara Kerja TanStack Query .....           | 36 |
| <b>Gambar 3.6</b>  | Wireframe Landing Page .....              | 37 |
| <b>Gambar 3.7</b>  | Wireframe Halaman Login .....             | 38 |
| <b>Gambar 3.8</b>  | Wireframe Halaman Register .....          | 38 |
| <b>Gambar 3.9</b>  | Wireframe Halaman Dashboard .....         | 39 |
| <b>Gambar 3.10</b> | Wireframe Halaman Sentiment .....         | 39 |
| <b>Gambar 3.11</b> | Wireframe Halaman Scraper .....           | 40 |
| <b>Gambar 3.12</b> | Wireframe Halaman Recommendation .....    | 40 |
| <b>Gambar 3.13</b> | Wireframe Halaman Chatbot .....           | 41 |

## DAFTAR TABEL

|                   |                                                               |    |
|-------------------|---------------------------------------------------------------|----|
| <b>Tabel 2.1</b>  | Tabel Perbandingan Penelitian Terkait .....                   | 22 |
| <b>Tabel 3.1</b>  | Jadwal Pelaksanaan Penelitian .....                           | 24 |
| <b>Tabel 3.2</b>  | Skenario Pengujian Landing Page .....                         | 43 |
| <b>Tabel 3.3</b>  | Skenario Pengujian Halaman Login .....                        | 43 |
| <b>Tabel 3.4</b>  | Skenario Pengujian Halaman Register .....                     | 45 |
| <b>Tabel 3.5</b>  | Skenario Pengujian Halaman Dashboard .....                    | 46 |
| <b>Tabel 3.6</b>  | Skenario Pengujian Halaman Dashboard Sentiment .....          | 48 |
| <b>Tabel 3.7</b>  | Skenario Pengujian Halaman Dashboard Rekomendasi Konten ..... | 49 |
| <b>Tabel 3.8</b>  | Skenario Pengujian Halaman Data Scraper .....                 | 50 |
| <b>Tabel 3.9</b>  | Skenario Pengujian Chatbot .....                              | 52 |
| <b>Tabel 3.10</b> | Skenario Pengujian Logout .....                               | 53 |

# BAB 1

## PENDAHULUAN

### 1.1 Latar Belakang

Di era digital saat ini, perilaku konsumen telah berubah secara signifikan. Media sosial seperti Instagram, tidak hanya menjadi saluran pemasaran, tetapi juga menjadi ruang interaksi antara konsumen dan pelaku usaha. Studi oleh (Trulline, 2021) menunjukkan bahwa UMKM semakin bergantung pada media sosial untuk mempromosikan produk, membangun reputasi, dan memahami minat pasar. Komentar konsumen pada postingan media sosial mencerminkan persepsi publik terhadap produk atau layanan UMKM, sehingga dapat dimanfaatkan sebagai indikator kepuasan dan reputasi merek.

Namun, data komentar media sosial bersifat tidak terstruktur, jumlahnya besar, dan berubah secara dinamis, sehingga sulit dianalisis tanpa bantuan teknologi. Sejalan dengan (Joseph, 2024), *Sentiment Analysis* pada media sosial merupakan bidang penelitian yang berkembang pesat dan melibatkan penggunaan *Natural Language Processing* (NLP), *text analysis*, dan *computational linguistics* untuk mengidentifikasi serta mengekstraksi informasi subjektif dari data teks. Pendekatan NLP tersebut memungkinkan komentar tidak terstruktur diproses menjadi kategori sentimen yang dapat diinterpretasikan, baik positif, negatif, maupun netral. Dengan demikian, hasil analisis sentimen dari proses NLP dapat disajikan dalam bentuk dashboard interaktif agar UMKM dapat memahami tren sentimen, isu yang sering muncul, serta persepsi pelanggan secara lebih cepat dan intuitif.

Dalam pengembangan dashboard analitik berbasis web, tantangan tidak hanya terletak pada penyajian visualisasi data, tetapi juga pada pengelolaan alur data di sisi *frontend*. Dashboard analitik merupakan aplikasi yang bersifat data-driven, di mana berbagai komponen antarmuka seperti grafik, tabel, dan indikator bergantung pada data yang sama yang bersumber dari dataset historis hasil scraping media sosial dan diproses atas permintaan pengguna. Jika pengambilan data dilakukan secara langsung pada setiap komponen tanpa arsitektur pengelolaan data yang terstruktur, maka dapat muncul berbagai permasalahan, seperti permintaan API berulang dan inkonsistensi data antar-komponen. Permasalahan tersebut tidak hanya berdampak pada aspek teknis pengembangan aplikasi, tetapi juga berimplikasi langsung terhadap pemanfaatan dashboard oleh pelaku UMKM, karena informasi yang ditampilkan berpotensi tidak akurat atau tidak konsisten antar-halaman, sehingga dapat menyebabkan interpretasi yang keliru terhadap tren sentimen dan kondisi pasar, menurunkan kepercayaan terhadap sistem, serta menurunkan

efektivitas pemanfaatan data sebagai dasar pengambilan keputusan yang berpotensi berdampak pada keberlanjutan usaha UMKM.

Pendekatan arsitektur *Client Data Layer* hadir sebagai solusi untuk mengelola data yang bersumber dari server secara terpusat di sisi klien. Dengan adanya *Client Data Layer*, proses pengambilan, *caching*, dan sinkronisasi data dapat dilakukan secara terstruktur, sehingga komponen antarmuka tidak perlu berinteraksi langsung dengan API. Salah satu pustaka yang mendukung pendekatan ini adalah TanStack Query, yang dirancang untuk mengelola server state secara efisien melalui mekanisme *caching*, deduplikasi permintaan, serta sinkronisasi data antar-komponen.

Sejumlah penelitian terdahulu menunjukkan bahwa TanStack Query memiliki keunggulan dalam pengelolaan data asinkron pada aplikasi *frontend* dibandingkan pendekatan pengelolaan data konvensional, khususnya pada aplikasi yang bersifat data-driven dan menampilkan data dalam jumlah besar (Luz, 2025). Penelitian yang membandingkan TanStack Query dengan pustaka state management lain juga melaporkan adanya peningkatan efisiensi pengelolaan server state dan pengurangan permintaan data yang tidak diperlukan (Micheal, 2025). Namun demikian, sebagian besar penelitian tersebut masih berfokus pada aspek teknis pustaka atau perbandingan antar-library, dan belum secara spesifik mengkaji penerapan TanStack Query sebagai *Client Data Layer* pada studi kasus dashboard analitik.

Berdasarkan permasalahan tersebut, penelitian ini muncul dari hipotesis bahwa penerapan arsitektur *Client Data Layer* menggunakan TanStack Query pada aplikasi dashboard analitik mampu menghasilkan pengelolaan data *frontend* yang lebih terstruktur dan konsisten dibandingkan pendekatan pengelolaan data konvensional, yaitu pengambilan data API secara langsung pada komponen antarmuka tanpa mekanisme pengelolaan data terpusat. Perilaku tersebut diasumsikan tercermin melalui pemanfaatan mekanisme *caching*, pengendalian permintaan data dari API, serta konsistensi data yang ditampilkan pada berbagai komponen dashboard. Untuk menguji hipotesis tersebut, TanStack Query diterapkan pada studi kasus pengembangan Dashboard Analisis Sentimen UMKM dengan metode *Fountain*, dan hasilnya dievaluasi melalui observasi perilaku sistem menggunakan pendekatan *blackbox testing*.

## **1.2 Rumusan Masalah**

1. Bagaimana menerapkan arsitektur *Client Data Layer* menggunakan TanStack Query pada pengembangan Dashboard Analisis Sentimen UMKM?
2. Bagaimana mekanisme pengelolaan data *frontend* yang dihasilkan setelah penerapan TanStack Query ditinjau mekanisme *caching*, dan konsistensi data antar-komponen dashboard?

### 1.3 Tujuan

1. Menerapkan arsitektur *Client Data Layer* menggunakan TanStack Query pada studi kasus Dashboard Analisis Sentimen UMKM.
2. Menganalisis mekanisme pengelolaan data *frontend* setelah penerapan TanStack Query, khususnya terkait konsistensi data, caching, dan permintaan API.

### 1.4 Manfaat

Manfaat Teoritis

1. Penelitian ini diharapkan dapat menambah literatur mengenai penerapan TanStack Query dalam arsitektur data layer pada studi kasus aplikasi dashboard.

Manfaat Praktis

#### 1. bagi UMKM

Penelitian ini menghasilkan sebuah dashboard analisis sentimen yang dapat membantu UMKM memahami persepsi konsumen berdasarkan data media sosial secara lebih terstruktur dan mudah dipahami.

#### 2. Manfaat bagi Pengembang

Penelitian ini memberikan studi kasus penerapan arsitektur *Client Data Layer* menggunakan TanStack Query yang dapat dijadikan acuan dalam merancang pengelolaan data *frontend* pada aplikasi data-driven.

#### 3. Manfaat bagi Akademisi

Penelitian ini dapat dijadikan referensi bagi penelitian sejenis yang membahas arsitektur *frontend*, pengelolaan server state, dan pengembangan dashboard analitik.

### 1.5 Batasan Masalah

Batasan proyek ditetapkan agar penelitian tetap terfokus pada tujuan yang telah dirumuskan. Adapun batasan proyek dalam Tugas Akhir ini adalah sebagai berikut:

1. Studi kasus penelitian difokuskan pada Dashboard Analisis Sentimen UMKM dengan ruang lingkup penelitian terbatas pada sisi *frontend* aplikasi.
2. Penelitian hanya membahas penerapan arsitektur *Client Data Layer* menggunakan TanStack Query pada aplikasi *frontend* berbasis React.
3. Proses analisis sentimen, termasuk pengumpulan data media sosial dan pemrosesan teks, tidak dibahas dalam penelitian ini dan sepenuhnya dilakukan pada sisi backend.
4. Penelitian ini tidak melakukan perbandingan implementasi TanStack Query dengan pustaka atau framework state management lain
5. Evaluasi sistem dilakukan menggunakan pendekatan *blackbox testing*, tanpa melakukan pengujian performa numerik atau *benchmarking* mendalam.

6. Penelitian ini tidak membahas aspek optimasi backend, keamanan aplikasi, pengujian beban, maupun pengujian kegunaan secara mendalam.

## **BAB 2**

### **TINJAUAN PUSTAKA**

#### **2.1 Landasan Teori**

##### **2.1.1 UMKM dan Digitalisasi**

Usaha Mikro, Kecil, dan Menengah (UMKM) merupakan sektor usaha produktif yang memiliki peran penting dalam perekonomian, khususnya dalam menciptakan lapangan kerja dan mendorong pertumbuhan ekonomi lokal. UMKM umumnya memiliki karakteristik berupa skala usaha yang relatif kecil, keterbatasan modal, serta pengelolaan usaha yang masih sederhana. Dalam beberapa tahun terakhir, perkembangan teknologi digital telah mendorong UMKM untuk beradaptasi dengan perubahan lingkungan bisnis agar tetap mampu bersaing di tengah dinamika pasar yang semakin cepat.

Meskipun digitalisasi menawarkan berbagai peluang, UMKM masih menghadapi sejumlah tantangan dalam proses adopsinya. Tantangan tersebut meliputi rendahnya literasi digital, keterbatasan pemahaman dalam pemanfaatan teknologi informasi, serta kurangnya kemampuan dalam mengelola dan menganalisis data bisnis. Selain itu, salah satu kendala utama yang dihadapi UMKM adalah keterbatasan akses terhadap sumber daya yang dibutuhkan untuk mengembangkan usaha, seperti modal, informasi, dan teknologi (Alviani et al., 2025).

Permasalahan digitalisasi UMKM juga diperkuat oleh faktor demografis dan geografis. Menurut (Sofyan and Agusman, 2025), rendahnya literasi digital, khususnya pada pelaku UMKM usia lanjut dan yang berada di wilayah terpencil, menjadi hambatan dalam proses transformasi digital. Selain itu, tidak semua UMKM memiliki perangkat pendukung dan akses jaringan internet yang memadai, sehingga proses digitalisasi belum dapat diterapkan secara merata.

Dalam konteks pemasaran, pemanfaatan teknologi digital, khususnya media sosial, telah menjadi salah satu sarana utama bagi UMKM untuk mempromosikan produk dan menjangkau konsumen secara lebih luas. Aktivitas pemasaran digital tersebut menghasilkan data interaksi konsumen dalam jumlah besar yang berpotensi memberikan informasi berharga mengenai persepsi dan preferensi pasar. Oleh karena itu, UMKM membutuhkan sistem informasi yang mampu mengolah dan menyajikan data tersebut secara terstruktur dan informatif. Keberadaan sistem informasi berbasis dashboard analitik menjadi penting untuk mendukung pengambilan keputusan berbasis data serta meningkatkan efektivitas strategi pemasaran UMKM di era digital (Trulline, 2021).

### **2.1.2 Media Sosial sebagai Sumber Data**

Media sosial telah berkembang tidak hanya sebagai sarana komunikasi dan pemasaran, tetapi juga sebagai sumber data yang mencerminkan opini, persepsi, dan perilaku konsumen. Melalui media sosial, konsumen dapat mengekspresikan pengalaman serta keterlibatan mereka terhadap suatu merek melalui berbagai bentuk interaksi, seperti komentar, unggahan, tanda suka, dan aktivitas berbagi konten. Interaksi tersebut mencerminkan keterlibatan konsumen dan menghasilkan data yang bernilai untuk dianalisis lebih lanjut (Mardhatilah et al., 2024). Data yang dihasilkan dari media sosial umumnya bersifat tidak terstruktur dan terus bertambah secara dinamis, sehingga memerlukan sistem yang mampu mengelola dan menyajikan informasi tersebut secara terstruktur agar dapat dimanfaatkan secara optimal.

Data yang dihasilkan dari media sosial memiliki karakteristik bersifat tidak terstruktur, berjumlah besar, dan terus bertambah secara dinamis. Komentar dan ulasan konsumen umumnya berbentuk teks bebas yang mengandung opini subjektif, emosi, serta penilaian terhadap suatu produk atau layanan. Kondisi ini menyebabkan data media sosial sulit dianalisis secara manual. Oleh karena itu, diperlukan pendekatan sistematis untuk mengolah dan mengekstraksi informasi penting dari data tersebut agar dapat dimanfaatkan secara optimal.

Dalam konteks UMKM, data media sosial berpotensi memberikan wawasan penting terkait preferensi konsumen, tingkat kepuasan pelanggan, serta isu-isu yang sering muncul dalam interaksi publik. Informasi ini dapat dimanfaatkan sebagai dasar evaluasi strategi pemasaran dan pengambilan keputusan bisnis. Namun, tanpa dukungan sistem yang mampu mengelola dan menyajikan data secara terstruktur, potensi data media sosial tersebut sulit dimanfaatkan secara efektif.

Oleh karena itu, media sosial diposisikan sebagai salah satu sumber data utama dalam pengembangan sistem analitik bagi UMKM. Data yang diperoleh dari media sosial selanjutnya dapat diolah dan disajikan dalam bentuk informasi yang lebih ringkas dan mudah dipahami melalui dashboard analitik. Pendekatan ini memungkinkan UMKM untuk memantau persepsi konsumen dan dapat digunakan sebagai alat pengambilan keputusan.

### **2.1.3 Analisis Sentimen pada Media Sosial**

Analisis sentimen merupakan pendekatan analitik yang digunakan untuk mengidentifikasi dan mengklasifikasikan opini atau sikap pengguna terhadap suatu objek, seperti produk, layanan, atau merek, berdasarkan data teks. Dalam konteks media sosial, analisis sentimen memanfaatkan komentar, ulasan, dan berbagai bentuk interaksi pengguna untuk menentukan kecenderungan sentimen yang umumnya dikategorikan ke dalam sentimen positif, negatif, atau netral (Suryani



and Komputer, 2024).

Seiring dengan meningkatnya aktivitas pengguna di media sosial, volume data opini yang dihasilkan semakin besar, bersifat dinamis, dan umumnya tidak terstruktur. Data tersebut mengandung unsur subjektivitas, emosi, serta bahasa informal, sehingga sulit dianalisis secara manual. Oleh karena itu, analisis sentimen digunakan untuk menyederhanakan data teks yang kompleks menjadi informasi yang lebih terstruktur agar dapat dimanfaatkan dalam memahami kecenderungan opini publik dan perilaku konsumen.

Penelitian oleh (Fajarini et al., 2025) menunjukkan bahwa analisis sentimen berbasis data media sosial mampu memberikan wawasan mengenai pola opini publik, preferensi konsumen, serta perubahan tren yang relevan dalam konteks bisnis. Pendekatan ini memungkinkan pemantauan opini dan dinilai lebih efisien dibandingkan metode konvensional, seperti survei manual atau riset pasar tradisional.

Selain itu, penelitian lain yang dilakukan oleh (Maharani, 2024) menegaskan bahwa pengolahan data media sosial melalui teknik data mining berperan penting dalam mengidentifikasi pola dan tren penggunaan data yang mencerminkan perilaku serta preferensi pengguna. Dalam konteks tersebut, analisis sentimen menjadi salah satu komponen yang berkontribusi dalam menggali makna dari data teks yang dihasilkan pengguna media sosial, sehingga wawasan yang diperoleh dapat dimanfaatkan untuk mendukung proses analisis dan pengambilan keputusan berbasis data.

Meskipun demikian, analisis sentimen juga memiliki keterbatasan, terutama terkait kualitas data yang bervariasi dan adanya noise dalam data media sosial yang dapat memengaruhi hasil interpretasi. Oleh karena itu, diperlukan pemahaman konteks dan pendekatan analisis yang tepat agar kesimpulan yang dihasilkan tidak bersifat keliru atau bias (Suryani and Komputer, 2024).

Dalam penelitian ini, analisis sentimen diposisikan sebagai proses pengolahan data pada sisi backend. Fokus penelitian tidak terletak pada metode atau algoritma analisis sentimen yang digunakan, melainkan pada pemanfaatan hasil analisis sentimen sebagai sumber data yang dikelola dan disajikan melalui dashboard analitik di sisi *frontend*.

#### **2.1.4 Dashboard Analitik dan Visualisasi Data**

Dashboard analitik merupakan sistem informasi yang dirancang untuk menyajikan data dalam bentuk visual yang ringkas, terintegrasi, dan mudah dipahami oleh pengguna. Dashboard ini umumnya memanfaatkan berbagai komponen visualisasi, seperti grafik, tabel, dan indikator kinerja, untuk menampilkan informasi penting yang mendukung proses pemantauan, analisis, dan pengambilan keputusan. Berbeda dengan laporan statis, dashboard analitik bersifat dinamis

dan interaktif, sehingga memungkinkan pengguna untuk memperoleh gambaran kondisi secara cepat dan menyeluruh.

Visualisasi data memiliki peran penting dalam dashboard analitik karena mampu mengubah data mentah menjadi informasi yang lebih bermakna dan intuitif. Melalui visualisasi yang tepat, pengguna dapat dengan mudah mengidentifikasi pola, tren, serta perubahan yang terjadi pada data. Visualisasi data juga berfungsi sebagai sarana penyampaian informasi yang bersifat naratif, di mana kinerja dan kondisi bisnis dapat digambarkan secara komprehensif tanpa harus melalui proses analisis data yang kompleks.

Penelitian yang dilakukan oleh (Rathore et al., 2025) menunjukkan bahwa penerapan teknik visualisasi data yang efektif dapat membantu pengambil keputusan dalam menghasilkan keputusan yang lebih informatif, akurat, dan ringkas. Studi tersebut menegaskan bahwa dashboard, berbagai jenis grafik, serta pendekatan visualisasi yang terstruktur berperan penting dalam meningkatkan kualitas *business intelligence* dan mendukung perencanaan strategis. Selain itu, visualisasi data dinilai mampu meningkatkan efisiensi proses pengambilan keputusan dan mengurangi waktu yang dibutuhkan untuk menganalisis data, sehingga organisasi dapat merespons permasalahan dan dinamika bisnis secara lebih cepat.

Pentingnya analisis dan visualisasi data dalam konteks bisnis juga dikemukakan oleh (Purnama et al., 2025), yang menyatakan bahwa visualisasi data berperan dalam membantu organisasi memahami tren dan pola yang terdapat dalam data. Informasi yang dihasilkan dari proses analisis tersebut dapat dimanfaatkan untuk mendukung pengambilan keputusan serta perencanaan bisnis berbasis data.

Dalam konteks aplikasi data-driven, dashboard analitik berfungsi sebagai jembatan antara data dan pengambilan keputusan. Dashboard memungkinkan penyajian data yang informatif sehingga perubahan kondisi dan tren dapat dipantau secara berkelanjutan. Hal ini menjadikan dashboard analitik sebagai komponen penting dalam sistem yang memanfaatkan data berskala besar dan bersifat dinamis, termasuk data yang dihasilkan dari media sosial.

Bagi UMKM, keberadaan dashboard analitik menjadi sangat relevan karena dapat menyederhanakan informasi yang kompleks menjadi tampilan visual yang mudah dipahami. Informasi seperti kecenderungan sentimen konsumen, pola interaksi pengguna, dan ringkasan data pemasaran dapat disajikan secara visual, sehingga pelaku UMKM tidak perlu melakukan analisis data secara manual. Dengan demikian, dashboard analitik dapat mendukung UMKM dalam mengambil keputusan bisnis yang lebih cepat, tepat, dan berbasis data.

### 2.1.5 *Blackbox Testing*

*Blackbox testing* merupakan salah satu metode pengujian perangkat lunak yang berfokus pada pengujian fungsional sistem dari sudut pandang pengguna tanpa memperhatikan struktur internal atau implementasi kode program. Pada metode ini, sistem diperlakukan sebagai sebuah “kotak hitam”, di mana pengujian dilakukan dengan memberikan input tertentu dan kemudian mengamati output atau respons yang dihasilkan untuk menilai kesesuaian fungsi sistem dengan spesifikasi yang telah ditetapkan. Pendekatan ini banyak digunakan dalam pengujian perangkat lunak karena mampu mengevaluasi perilaku sistem secara langsung berdasarkan kebutuhan pengguna.

Metode *blackbox testing* menitikberatkan pada validasi fungsi utama sistem, seperti proses input dan output, alur penggunaan, serta respons sistem terhadap berbagai kondisi penggunaan. Dengan demikian, metode ini sangat sesuai untuk menguji sistem berbasis antarmuka pengguna, khususnya aplikasi web dan dashboard, yang keberhasilannya sangat bergantung pada kesesuaian fungsi dan kemudahan penggunaan. Pengujian dilakukan tanpa melibatkan analisis terhadap logika internal atau struktur kode, sehingga hasil pengujian lebih berorientasi pada pengalaman dan kebutuhan pengguna akhir.

Penelitian yang dilakukan oleh Maulida et al. (Maulida et al., 2025) menunjukkan bahwa *blackbox testing* efektif digunakan dalam pengujian sistem website pemesanan online karena mampu mengidentifikasi kesalahan fungsional pada tahap awal pengembangan. Dalam penelitian tersebut, pengujian dilakukan pada berbagai fitur utama sistem, seperti proses registrasi, login, pencarian produk, hingga konfirmasi pesanan, tanpa meninjau kode program yang digunakan. Hasil penelitian tersebut membuktikan bahwa penerapan *blackbox testing* dapat membantu memastikan kualitas sistem dari aspek fungsionalitas sebelum sistem dirilis kepada pengguna.

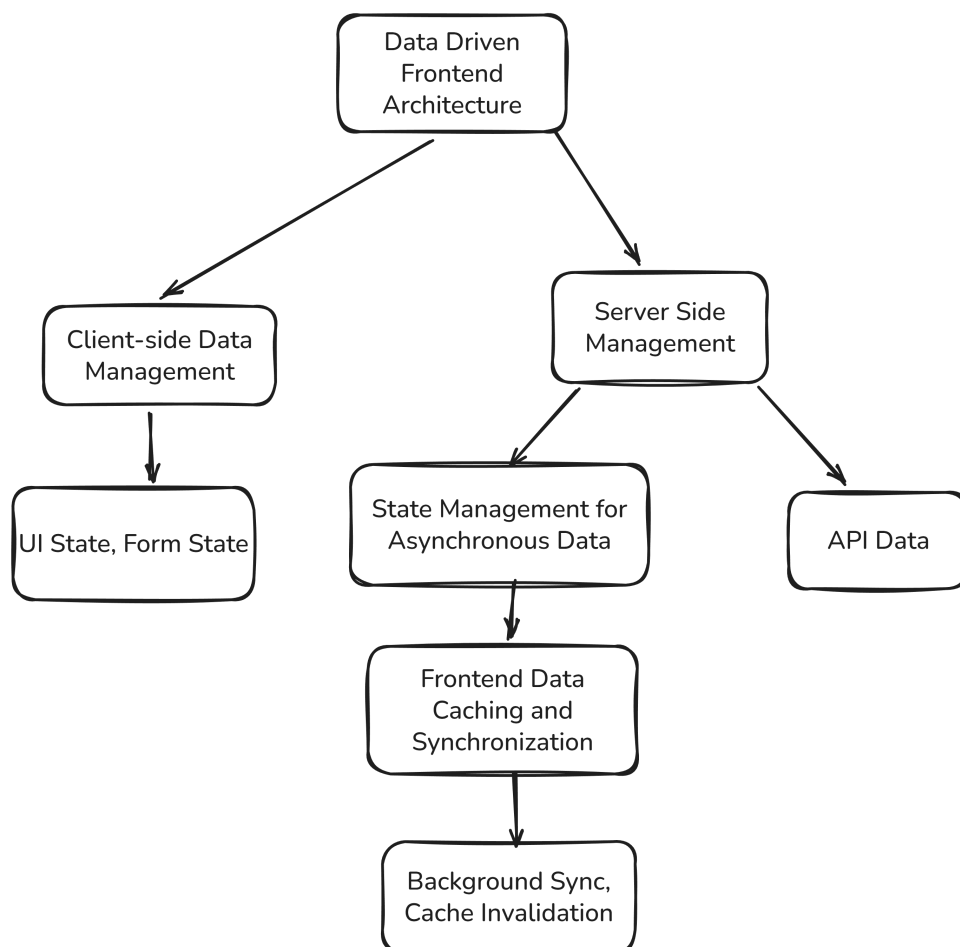
Berdasarkan karakteristik tersebut, black-box testing dipandang sebagai metode pengujian yang relevan untuk digunakan dalam penelitian ini. Metode ini memungkinkan evaluasi terhadap perilaku sistem *frontend* secara menyeluruh berdasarkan skenario penggunaan, sehingga kesesuaian fungsi sistem dengan kebutuhan pengguna dapat dinilai secara sistematis.

### 2.1.6 *Arsitektur Client Data Layer*

Pada aplikasi *frontend* modern yang bersifat data-driven, pengelolaan data yang bersumber dari Application Programming Interface (API) menjadi salah satu aspek penting dalam arsitektur sistem. Aplikasi seperti dashboard analitik umumnya menampilkan berbagai komponen antarmuka, seperti grafik, tabel, dan indikator, yang bergantung pada data yang sama dan diperbarui berdasarkan sumber data dari API. Jika pengambilan dan pengolahan data

dilakukan secara langsung di setiap komponen antarmuka, maka dapat menimbulkan berbagai permasalahan, seperti permintaan API yang berulang, inkonsistensi data antar-komponen, serta peningkatan beban render yang berdampak pada penurunan performa aplikasi.

*Client Data Layer* merupakan pendekatan arsitektural dalam pengembangan *frontend* modern yang bertujuan untuk mengelola data yang bersumber dari server secara terpusat di sisi klien. Pendekatan ini muncul sebagai respons terhadap meningkatnya kompleksitas aplikasi data-driven, dimana data bersifat asinkron, dinamis, dan digunakan oleh banyak komponen antarmuka secara bersamaan. Dengan adanya *Client Data Layer*, proses *fetching*, *caching*, dan *synchronize data* dapat dilakukan secara lebih terstruktur, sehingga membantu menjaga konsistensi data dan mendukung pengelolaan data *frontend* secara lebih terstruktur.



**Gambar 2.1** Arsitektur *Client Data Layer*

#### 1. *Data-driven Frontend Architecture*

*Data-driven Frontend Architecture* merupakan pendekatan pengembangan di mana seluruh antarmuka pengguna didorong oleh data sebagai sumber kebenaran utama. Dalam pola ini, UI dihasilkan sebagai fungsi dari state yang ada—artinya, setiap perubahan pada data akan secara otomatis memicu pembaruan pada tampilan aplikasi.

Arsitektur ini sangat berguna untuk aplikasi *frontend* yang menampilkan informasi dinamis seperti dashboard analitik, papan monitoring. Keunggulan utamanya adalah konsistensi tampilan yang lebih terjamin, alur data yang mudah dilacak, dan pengembangan fitur baru yang lebih sistematis karena UI dan logika data terpisah dengan jelas.

## 2. *Server State Management*

*Server state management* mengacu pada pengelolaan data yang bersumber dari sistem eksternal, seperti API atau layanan backend, yang bersifat asinkron dan dapat berubah di luar kendali langsung aplikasi klien. Data ini memiliki karakteristik dinamis karena dipengaruhi oleh kondisi jaringan, waktu respons server, serta pembaruan data di sisi backend. Oleh karena itu, server state memerlukan mekanisme khusus untuk menangani proses pengambilan data, status pemuatan, penanganan kesalahan, serta pembaruan dan sinkronisasi data agar informasi yang digunakan oleh berbagai komponen antarmuka tetap konsisten dan akurat.

## 3. *Client-side Data Management*

*Client-side data management* berfokus pada pengelolaan data di sisi klien setelah data tersebut diperoleh dari *server*, termasuk penyimpanan sementara, penggunaan ulang data, serta distribusi data ke berbagai komponen antarmuka. Pendekatan ini bertujuan untuk mengurangi ketergantungan terhadap permintaan data berulang ke server sekaligus menjaga konsistensi informasi yang ditampilkan pada antarmuka pengguna. Dengan pengelolaan data yang terstruktur di sisi klien, aplikasi *frontend* dapat menyajikan data secara lebih responsif dan stabil, khususnya pada aplikasi *frontend* yang bersifat data-driven seperti dashboard analitik, tanpa mencampurkan pengelolaan data server dengan state antarmuka pengguna.

## 4. *State Management for Asynchronous Data*

*State Management for Asynchronous Data* adalah pendekatan khusus untuk menangani data yang diperoleh melalui state, seperti panggilan API, operasi file, atau permintaan jaringan lainnya. Karena data tersebut tidak tersedia secara instan, sistem harus mampu mengelola berbagai state yang mungkin terjadi: mulai dari state *idle*, state *fetching* atau *onloading*, state *success*, hingga state *error*. Tantangan utamanya adalah memastikan aplikasi tetap responsif dan memberikan umpan balik yang informatif kepada pengguna selama proses pengambilan data. Pendekatan ini juga mencakup strategi seperti pembatalan permintaan yang tidak diperlukan, pengulangan otomatis saat gagal, dan pembaruan data latar belakang untuk menjaga

informasi tetap konsisten.

## 5. *Frontend Data Caching and Synchronization*

*Frontend Data Caching and Synchronization* adalah teknik untuk meningkatkan kinerja aplikasi dengan menyimpan salinan data dari server di memori klien. Dengan adanya *cache*, aplikasi dapat menampilkan informasi secara cepat tanpa perlu melakukan permintaan berulang ke server. Namun, teknik ini juga menimbulkan tantangan, yaitu data yang disimpan dapat menjadi kedaluwarsa jika terjadi perubahan di sisi server. Oleh karena itu, diperlukan mekanisme sinkronisasi yang cerdas, seperti pembaruan di latar belakang (background refresh) dan penandaan *cache* yang kedaluwarsa (cache invalidation). Dengan pendekatan ini, aplikasi dapat menampilkan data secara lebih konsisten sekaligus menjaga keakuratan informasi yang ditampilkan.

Beberapa penelitian menunjukkan bahwa penerapan mekanisme *caching* pada aplikasi *frontend* dapat meningkatkan performa dan responsivitas sistem. *Caching* memungkinkan data hasil pemanggilan API disimpan sementara di sisi klien sehingga dapat digunakan kembali tanpa melakukan permintaan ulang ke server. Pendekatan ini terbukti mampu mengurangi duplikasi permintaan API dan mempercepat waktu respons aplikasi. Pemanfaatan pustaka seperti TanStack Query dalam mengelola *caching* dan prefetching data juga dinilai efektif dalam meningkatkan efisiensi pengelolaan data di sisi *frontend* serta pengalaman pengguna secara keseluruhan (Rahman and Prihanto, 2024). Dalam konteks penelitian ini, peningkatan performa dipahami sebagai perubahan mekanisme pengelolaan data *frontend* yang diamati melalui mekanisme *caching* dan pengendalian permintaan data, tanpa dilakukan pengukuran performa numerik.

Melalui fitur-fitur seperti *smart caching*, *refetching*, dan dukungan terhadap penanganan data asinkron, pendekatan ini membantu menyederhanakan kompleksitas pengelolaan data yang bersumber dari API serta mendukung peningkatan efisiensi pengelolaan data di sisi frontend. Selain itu, dukungan terhadap TypeScript memungkinkan konsistensi struktur data tetap terjaga tanpa mencampurkan pengelolaan server state dengan *client state*, sehingga sesuai dengan kebutuhan aplikasi *frontend* modern yang bersifat data-driven (Highlights, 2025).

Perlu dibedakan secara konseptual antara *client state* dan *server state* dalam pengembangan aplikasi *frontend* modern. *Client state* mengacu pada state yang sepenuhnya berada di sisi klien dan berkaitan langsung dengan antarmuka pengguna, seperti status tampilan,

input formulir, filter, atau kondisi navigasi. *Server state* merepresentasikan data yang bersumber dari sistem eksternal seperti API atau layanan backend, bersifat asinkron, dapat berubah di luar kendali aplikasi klien, serta memerlukan penanganan khusus terkait proses pengambilan data, status pemuatan, penanganan kesalahan, dan sinkronisasi.

Dalam konteks pengembangan aplikasi frontend yang bersifat data-driven, istilah data frontend terstruktur pada penelitian ini merujuk pada cara data yang bersumber dari server dikelola, didistribusikan, dan digunakan di sisi klien. Data yang dikelola melalui mekanisme terpusat di sisi *frontend* memiliki bentuk dan siklus hidup yang konsisten, serta dapat digunakan ulang oleh berbagai komponen antarmuka tanpa menimbulkan inkonsistensi informasi maupun permintaan data yang tidak terkontrol. Definisi ini selaras dengan prinsip *frontend state management* yang dikemukakan oleh (Evergreen, 2023), di mana keterstruktur data ditentukan oleh bagaimana data dikendalikan, dipertahankan, dan disinkronkan di dalam aplikasi.

Berdasarkan definisi tersebut, pengelolaan data *frontend* dapat dikatakan terstruktur apabila memenuhi beberapa kriteria berikut:

1. Pengelolaan Data Terpusat (*Single Source of Truth*)

Data yang bersumber dari server dikelola melalui satu lapisan pengelolaan data di sisi *frontend*, sehingga komponen antarmuka tidak melakukan pengambilan data secara langsung ke API. Pendekatan ini menghindari duplikasi data dan memastikan seluruh komponen menggunakan sumber data yang sama, sehingga menciptakan prediksi yang lebih baik terhadap perubahan state (Evergreen, 2023).

2. Konsistensi Bentuk dan Representasi Data

Data yang digunakan oleh komponen antarmuka memiliki struktur dan representasi yang konsisten, baik dari segi skema, *interface* maupun nilai yang ditampilkan. Dengan konsistensi ini, perbedaan tampilan informasi antar-komponen dapat dihindari meskipun data digunakan pada konteks visualisasi yang berbeda.

3. Siklus Hidup Data

Data *frontend* memiliki status yang terdefinisi dengan jelas, seperti *loading*, *success*, dan *error*. Kejelasan siklus hidup ini memungkinkan sistem untuk memberikan umpan balik yang tepat kepada pengguna serta memudahkan pengelolaan data yang bersifat asinkron, termasuk klasifikasi state berdasarkan umur panjangnya (*persistent vs non-persistent*) (Evergreen, 2023).

4. Pemanfaatan Mekanisme *Caching*

Data hasil pengambilan dari server disimpan sementara di sisi frontend dan dapat digunakan kembali oleh komponen lain tanpa harus melakukan permintaan ulang

ke API. Mekanisme *caching* berperan penting dalam mengurangi permintaan data yang berulang serta meningkatkan efisiensi pengelolaan data, sekaligus mendukung persistensi data untuk menghindari kehilangan informasi saat reload (Evergreen, 2023).

#### 5. Sinkronisasi Data Antar-Komponen

Perubahan data yang terjadi akibat pembaruan atau refetch dapat tercermin secara otomatis pada seluruh komponen yang menggunakan data tersebut. Dengan adanya sinkronisasi, informasi yang ditampilkan pada dashboard tetap konsisten meskipun terjadi interaksi pengguna atau navigasi antar-halaman, sehingga mengoptimalkan performa seperti minimisasi re-render yang tidak perlu (Evergreen, 2023).

#### 6. *Readability* (Keterbacaan Struktur)

Sistem pengelolaan data harus terorganisir dengan baik untuk memudahkan pemeliharaan dan pengembangan, menghindari redundansi data atau modifikasi state yang tidak perlu, sehingga meningkatkan kemudahan kolaborasi antar-komponen (Evergreen, 2023).

#### 7. *Scalability* (Skalabilitas)

Pengelolaan data harus mendukung pertumbuhan aplikasi dengan menangani peningkatan volume data dan state secara efisien, tanpa membatasi ekstensi fitur atau menimbulkan *bottleneck* performa (Evergreen, 2023).

Pengelolaan data *frontend* yang terstruktur menuntut adanya mekanisme pengelolaan *server state* yang mampu mengatur proses pengambilan, penyimpanan sementara, dan pembaruan data secara konsisten. Dalam konteks ini, pemanfaatan mekanisme *caching* memungkinkan data yang diperoleh dari server digunakan ulang oleh berbagai komponen tanpa menimbulkan permintaan API berulang, sekaligus tetap mendukung pembaruan data ketika diperlukan. Selain itu, pengelolaan data secara terpusat dan adanya mekanisme sinkronisasi memastikan konsistensi data antar-komponen, sehingga seluruh antarmuka menampilkan informasi yang selaras. Dengan demikian, keterstrukturan data *frontend* dalam penelitian ini dipahami sebagai integrasi antara pengelolaan *server state*, penerapan *caching*, dan sinkronisasi data antar-komponen, yang menjadi fondasi penerapan arsitektur *Client Data Layer* untuk menghasilkan pengelolaan data *frontend* yang konsisten, dan efisien.

Pengelolaan data *frontend* yang tidak terstruktur terjadi ketika data yang bersumber dari server dikelola secara langsung oleh masing-masing komponen antarmuka tanpa mekanisme pengelolaan terpusat di sisi frontend. Pada pendekatan ini, proses pengambilan, penyimpanan, dan pembaruan data dilakukan secara terpisah di setiap komponen. Akibatnya, data tidak



memiliki siklus hidup yang konsisten, seperti status loading, success, dan error dikelola secara terpisah, serta data yang sama berpotensi diambil ulang dari API dan tidak adanya mekanisme *caching*.

Kondisi tersebut berdampak langsung pada inkonsistensi data antar-komponen, karena setiap komponen dapat menampilkan data pada waktu dan kondisi pembaruan yang berbeda. Pada aplikasi dashboard analitik, hal ini berpotensi menimbulkan perbedaan informasi antar-komponen atau antar-halaman dan mengurangi tingkat kepercayaan terhadap sistem sebagai alat bantu pengambilan keputusan berbasis data. Oleh karena itu, keterbatasan pengelolaan data langsung di tingkat komponen menjadi dasar perlunya penerapan arsitektur *Client Data Layer* untuk mencapai pengelolaan data *frontend* yang lebih terstruktur, konsisten, dan efisien.

### **2.1.7 REST API**

*Representational State Transfer Application Programming Interface (REST API)* merupakan gaya arsitektur layanan web yang digunakan untuk memungkinkan komunikasi antara klien dan server melalui protokol HTTP secara terstandarisasi. REST API bersifat stateless, di mana setiap permintaan dari klien harus membawa seluruh informasi yang dibutuhkan untuk diproses oleh server, sehingga tidak bergantung pada status permintaan sebelumnya. Data yang dipertukarkan umumnya disajikan dalam format JSON karena bersifat ringan dan mudah diproses oleh aplikasi *frontend*, menjadikan REST API banyak digunakan pada aplikasi web modern yang bersifat data-driven.

Dalam konteks aplikasi *frontend analitik*, REST API berperan sebagai sumber utama data (server state) yang dikonsumsi oleh antarmuka pengguna. Data yang diperoleh melalui REST API bersifat asinkron dan dapat berubah sewaktu-waktu, sehingga memerlukan mekanisme pengelolaan data yang mampu menangani proses pengambilan, pembaruan, dan sinkronisasi data secara efisien. Penelitian terkini menunjukkan bahwa REST API memiliki keunggulan dalam penyajian data yang bersifat datar dan mudah di-cache, sehingga mampu meningkatkan efisiensi distribusi data serta memperbesar rasio cache hit pada lapisan jaringan. Pendekatan ini dinilai lebih optimal untuk kebutuhan aplikasi yang menampilkan data terstruktur secara berulang, seperti dashboard dan sistem pelaporan, dibandingkan pendekatan API lain yang lebih kompleks (Islam, 2025).

Lebih lanjut, penelitian tersebut menegaskan bahwa performa aplikasi web tidak ditentukan oleh satu teknologi tertentu, melainkan oleh keselarasan antara desain akses data, mekanisme *caching*, serta pengelolaan state pada sisi klien. REST API yang dirancang dengan kontrak data yang jelas dan *cache-aware* terbukti mendukung peningkatan performa dan skalabilitas aplikasi ketika dipadukan dengan lapisan pengelolaan data di *frontend*. Oleh karena

itu, dalam pengembangan aplikasi *frontend* modern, REST API umumnya tidak diakses secara langsung oleh setiap komponen antarmuka, melainkan melalui lapisan pengelolaan data seperti *Client Data Layer* agar data dapat dikelola secara terpusat, konsisten, dan efisien.

Dalam penelitian ini, REST API diposisikan sebagai penyedia data hasil analisis sentimen yang diproses di sisi backend. Data tersebut selanjutnya dikelola pada sisi frontend melalui arsitektur *Client Data Layer* sebelum ditampilkan dalam bentuk visualisasi pada dashboard analitik. Dengan pemisahan peran ini, REST API berfungsi sebagai sumber data, sementara pengelolaan performa, *caching*, dan sinkronisasi data dilakukan sepenuhnya di sisi *frontend* untuk mendukung penyajian informasi yang responsif dan konsisten.

Penerapan *Client Data Layer* memberikan sejumlah manfaat dalam pengembangan aplikasi *frontend*. Salah satu manfaat utama adalah peningkatan konsistensi data, di mana beberapa komponen yang membutuhkan data yang sama dapat memperoleh informasi yang seragam tanpa harus melakukan permintaan data secara terpisah. Selain itu, *Client Data Layer* memungkinkan pengurangan jumlah permintaan API yang tidak diperlukan melalui mekanisme *caching* dan pengelolaan siklus data. Pendekatan ini juga mendukung penyajian data yang lebih stabil dan terkelola serta mempermudah pengelolaan data yang bersifat asinkron dan dinamis.

Dalam konteks dashboard analitik, keberadaan *Client Data Layer* menjadi semakin penting karena data yang ditampilkan umumnya bersifat besar, sering diperbarui, dan digunakan oleh banyak komponen secara bersamaan. Dengan memanfaatkan *Client Data Layer*, dashboard dapat menampilkan data secara lebih responsif dan stabil, sekaligus meminimalkan risiko inkonsistensi informasi yang ditampilkan kepada pengguna. Pendekatan ini mendukung terciptanya arsitektur *frontend* yang lebih terorganisasi, mudah dipelihara, dan skalabel.

### **2.1.8 React**

React merupakan sebuah pustaka (*library*) JavaScript yang digunakan untuk membangun antarmuka pengguna (user interface) pada aplikasi web yang bersifat interaktif dan responsif. Menurut dokumentasi resmi React, pustaka ini dirancang untuk membangun antarmuka dengan pendekatan *component-based*, di mana tampilan antarmuka dibagi menjadi bagian-bagian kecil yang dapat digunakan kembali (*reusable components*) sehingga memudahkan pengelolaan dan pemeliharaan kode aplikasi (React, 2026). React bekerja dengan memanfaatkan virtual DOM untuk meminimalkan operasi pada DOM aktual sehingga perubahan tampilan dapat dilakukan secara efisien saat data atau state aplikasi berubah, tanpa perlu melakukan refresh seluruh halaman.

Pendekatan *component-based architecture* yang digunakan React juga memungkinkan pengembang merancang antarmuka sebagai susunan komponen modular yang saling terpisah

namun saling berinteraksi, yang selaras dengan prinsip pengembangan frontend modern. Struktur seperti ini tidak hanya meningkatkan keterbacaan dan modularitas kode, tetapi juga memberikan fleksibilitas dalam pengembangan aplikasi berskala besar serta mempermudah pengujian dan pemeliharaan. Dalam konteks pengembangan aplikasi *Single Page Application (SPA)* dan *API-driven application*, studi yang dilakukan pada tren pengembangan aplikasi web menunjukkan bahwa penggunaan React dalam kombinasi dengan arsitektur API terbukti mendukung pengembangan aplikasi web yang modular, fleksibel, dan mudah dikembangkan secara berkelanjutan (Putra et al., 2025).

Dengan karakteristik tersebut, React dipilih sebagai teknologi *frontend* dalam penelitian ini untuk membangun antarmuka pengguna yang dinamis dan dapat berinteraksi secara langsung dengan lapisan pengelolaan data (*Client Data Layer*), sehingga mendukung kebutuhan aplikasi yang bersifat *data-driven*.

### **2.1.9 TanStack Query (React Query)**

TanStack Query merupakan pustaka manajemen data pada sisi *frontend* yang dirancang untuk mengelola data yang bersumber dari server (*server state*) secara efisien. Dalam dokumentasi resminya, TanStack Query dijelaskan sebagai "*the missing data-fetching layer for web applications*" yang berfungsi untuk mempermudah proses pengambilan, penyimpanan sementara (*caching*), sinkronisasi, serta pembaruan data dari server (Tanstack LCC, 2025). Pendekatan ini ditujukan untuk menangani kompleksitas data asinkron yang tidak dapat dikelola secara optimal menggunakan mekanisme state management konvensional.

Dalam dokumentasi resminya, TanStack Query mendefinisikan *server state* sebagai data yang berasal dari sumber eksternal dan memiliki karakteristik asinkron, dapat berubah sewaktu-waktu, serta memerlukan mekanisme khusus untuk menjaga konsistensi data di sisi klien. Oleh karena itu, TanStack Query menyediakan pendekatan deklaratif dalam pengelolaan *server state*, di mana pengembang dapat mendefinisikan kebutuhan data tanpa harus menangani secara manual proses sinkronisasi dan pembaruan data di setiap komponen antarmuka (TanStack Documentation, 2024).

Salah satu fitur utama TanStack Query adalah mekanisme *caching* yang memungkinkan data hasil pemanggilan API disimpan sementara di sisi klien. Dengan adanya *caching*, data yang telah diperoleh dapat digunakan kembali oleh komponen lain tanpa perlu melakukan permintaan ulang ke server, selama data tersebut masih dianggap valid. Pendekatan ini berkontribusi dalam mengurangi jumlah permintaan API yang tidak diperlukan, meningkatkan efisiensi aplikasi, serta mempercepat waktu respons antarmuka pengguna.

Selain *caching*, TanStack Query juga menyediakan mekanisme sinkronisasi data yang

mendukung pembaruan data secara otomatis. Melalui konsep seperti refetching dan invalidasi data, TanStack Query memastikan bahwa data yang ditampilkan tetap mutakhir ketika terjadi perubahan di sisi server. Mekanisme ini sangat relevan pada aplikasi data-driven, seperti dashboard analitik, yang menampilkan data secara dinamis dan digunakan oleh banyak komponen secara bersamaan.

Dalam konteks arsitektur *frontend*, TanStack Query dapat diposisikan sebagai implementasi konkret dari *Client Data Layer*. Pustaka ini berperan sebagai lapisan perantara antara backend API dan komponen antarmuka pengguna, sehingga komponen UI tidak berinteraksi langsung dengan API. Dengan demikian, TanStack Query membantu memisahkan logika pengelolaan data dari logika tampilan, meningkatkan keterbacaan kode, serta mempermudah pemeliharaan aplikasi dalam jangka panjang.

Pada penelitian ini, TanStack Query digunakan sebagai solusi untuk menerapkan arsitektur *Client Data Layer* pada pengembangan dashboard analitik. Fokus penggunaan TanStack Query diarahkan pada pengelolaan server state, caching data, serta sinkronisasi data antar-komponen, tanpa membahas aspek internal pustaka atau detail implementasi secara mendalam. Dengan pendekatan ini, TanStack Query berperan sebagai fondasi pengelolaan data pada sisi *frontend* yang mendukung penyajian informasi sentimen secara konsisten, responsif, dan efisien.

### **2.1.10 Metode Fountain**

Metode Fountain merupakan salah satu model dalam *Software Development Life Cycle (SDLC)* yang bersifat iteratif dan fleksibel, sebagai alternatif terhadap model pengembangan linear seperti Waterfall. Model ini memungkinkan fase-fase pengembangan seperti analisis kebutuhan, perancangan, implementasi, dan pengujian untuk saling tumpang tindih serta dilakukan ulang sesuai kebutuhan proyek, sehingga proses pengembangan dapat menyesuaikan perubahan persyaratan yang muncul sepanjang siklus hidup sistem.

Selain bersifat iteratif, metode Fountain menekankan kelanjutan antar-tahapan pengembangan tanpa batasan transisi yang kaku. Setiap aktivitas dalam siklus pengembangan tidak dipandang sebagai tahap yang berdiri sendiri. Dengan pendekatan ini, umpan balik yang diperoleh pada tahap implementasi atau pengujian dapat segera digunakan untuk memperbaiki hasil analisis maupun perancangan sebelumnya. Hal tersebut memungkinkan kesalahan desain atau ketidaksesuaian kebutuhan terdeteksi lebih awal, sehingga risiko *rework* pada tahap akhir dapat diminimalkan.

Fleksibilitas metode Fountain menjadikannya relevan untuk pengembangan sistem yang bersifat dinamis, seperti aplikasi dashboard analitik. Pada sistem semacam ini, perubahan

kebutuhan pengguna, penyesuaian arsitektur sering kali muncul selama proses pengembangan berlangsung. Oleh karena itu, model Fountain memberikan kerangka kerja yang adaptif namun tetap terstruktur, sehingga pengembangan sistem dapat berjalan secara terkendali tanpa kehilangan kemampuan untuk beradaptasi terhadap perubahan yang terjadi.

### Gambar 2.2 Fountain SDLC Model

Gambar 2.2 menunjukkan ilustrasi model Fountain, di mana fase-fase pengembangan tidak lagi bergerak secara kaku dari satu tahap ke tahap berikutnya, tetapi fase dapat saling berinteraksi dan kembali ke fase sebelumnya bila terdapat kebutuhan penyesuaian. Karakteristik ini memungkinkan proses pengembangan sistem menjadi lebih adaptif terhadap perubahan yang tidak terduga, sehingga risikonya dapat diminimalkan dan kualitas akhir sistem meningkat.

sifatnya yang fleksibel dan iteratif, metode ini dinilai sesuai untuk penelitian rekayasa perangkat lunak yang membutuhkan penyesuaian desain dan pengambilan keputusan teknis secara berulang tanpa mengganggu keseluruhan alur pengembangan sistem .

### **Tahapan Metode Fountain**

Metode Fountain terdiri dari beberapa tahapan pengembangan perangkat lunak yang dilakukan secara iteratif dan fleksibel. Setiap tahapan tidak bersifat kaku dan dapat saling tumpang tindih, sehingga memungkinkan penyesuaian kembali ke tahapan sebelumnya apabila ditemukan perubahan kebutuhan atau permasalahan selama proses pengembangan. Secara umum, tahapan dalam metode Fountain meliputi analisis, requirement specification, design, coding, testing, operation, maintenance, dan Evolution Siva et al. (2023).

Tahap *analysis* merupakan tahapan awal yang bertujuan untuk memahami permasalahan dan kebutuhan sistem secara umum. Pada tahap ini dilakukan identifikasi kebutuhan pengguna, ruang lingkup sistem, serta permasalahan yang ingin diselesaikan melalui pengembangan perangkat lunak. Hasil dari tahap analisis menjadi dasar bagi tahapan-tahapan selanjutnya.

Tahap *requirement specification* berfokus pada perumusan kebutuhan sistem secara lebih terstruktur dan terdokumentasi. Kebutuhan tersebut mencakup kebutuhan fungsional dan non-fungsional yang harus dipenuhi oleh sistem. Spesifikasi kebutuhan berperan penting sebagai acuan dalam proses perancangan dan implementasi sistem.

Tahap *design* merupakan tahapan perancangan solusi berdasarkan spesifikasi kebutuhan yang telah ditetapkan. Pada tahap ini dilakukan perancangan arsitektur sistem, struktur data, serta rancangan antarmuka pengguna. Hasil perancangan bertujuan untuk memberikan gambaran teknis mengenai bagaimana sistem akan dibangun sebelum masuk ke tahap implementasi.

Tahap *coding* adalah tahapan implementasi dari desain yang telah dibuat ke dalam bentuk kode program. Pada tahap ini, pengembang menerjemahkan rancangan sistem menjadi perangkat lunak yang dapat dijalankan sesuai dengan kebutuhan yang telah ditentukan.

Tahap *testing* dilakukan untuk memastikan bahwa sistem yang dikembangkan telah berjalan sesuai dengan spesifikasi dan bebas dari kesalahan fungsional. Pengujian dilakukan untuk memverifikasi fungsi-fungsi sistem serta memastikan bahwa sistem dapat digunakan dengan baik oleh pengguna.

Tahap *operation* merupakan tahapan di mana sistem telah siap digunakan dalam lingkungan operasional. Pada tahap ini, sistem mulai diakses oleh pengguna dan digunakan untuk mendukung aktivitas yang telah dirancang.

Tahap *maintenance* bertujuan untuk menjaga kinerja sistem setelah digunakan secara operasional. Aktivitas pada tahap ini meliputi perbaikan kesalahan, penyesuaian terhadap

perubahan lingkungan, serta peningkatan minor terhadap sistem agar tetap berjalan dengan optimal.

Tahap *Evolution* merupakan tahapan pengembangan lanjutan dalam metode Fountain yang bertujuan untuk melakukan peningkatan dan penambahan fitur pada sistem berdasarkan hasil penggunaan dan kebutuhan yang muncul. Tahapan ini memastikan sistem dapat terus dikembangkan secara berkelanjutan agar tetap sesuai dengan kebutuhan pengguna dan tujuan pengembangan

### **Alasan Pemilihan Metode Fountain**

Pemilihan metode *Software Development Life Cycle (SDLC)* yang tepat merupakan langkah strategis yang krusial untuk memastikan keberhasilan proyek serta mencegah pembengkakan biaya dan waktu pengembangan (Aniley et al., 2024). Berdasarkan pertimbangan tersebut, penelitian ini mengadopsi metode Fountain yang dinilai relevan karena memiliki karakteristik fleksibel namun tetap terstruktur. Efektivitas metode ini didukung oleh penelitian (Sastra and Sutawinata, 2023) pada perancangan sistem SIP-PTK, yang menunjukkan bahwa model Fountain mampu memandu tahapan analisis, desain, implementasi, hingga pengujian secara efektif pada sistem yang memiliki kebutuhan pengolahan data yang spesifik.

Relevansi metode Fountain juga sejalan dengan fokus penelitian ini, yaitu pengembangan aplikasi *frontend* yang bersifat data-driven dan memiliki ketergantungan tinggi terhadap interaksi antar-komponen. Dalam pengembangan *frontend*, tahapan perancangan arsitektur, implementasi, dan pengujian tidak berjalan secara linier, melainkan saling berkaitan dan sering memerlukan penyesuaian berdasarkan hasil evaluasi sistem. Penerapan metode Fountain memungkinkan tahapan pengembangan dan evaluasi berjalan secara paralel serta saling tumpang tindih (*overlapping*) tanpa menghilangkan struktur dan urutan penelitian yang jelas.

Melalui pendekatan ini, aktivitas analisis kebutuhan, perancangan arsitektur *frontend*, implementasi *Client Data Layer*, serta pengujian sistem dapat dilakukan secara berulang dan saling mempengaruhi. Hasil evaluasi pada satu tahapan dapat langsung digunakan sebagai dasar penyesuaian pada tahapan lainnya, seperti perubahan struktur data atau mekanisme sinkronisasi, tanpa harus menunggu seluruh siklus pengembangan selesai. Dengan demikian, proses pengembangan sistem menjadi lebih adaptif dan terkontrol, sehingga diharapkan mampu menghasilkan hasil penelitian yang sesuai dengan tujuan penelitian secara optimal.

## **2.2 Penelitian Terkait**

Berikut adalah tabel perbandingan penelitian terkait yang relevan dengan pengembangan penerapan arsitektur *Client Data Layer* menggunakan TanStack Query pada dashboard analisis sentimen

**Tabel 2.1** Tabel Perbandingan Penelitian Terkait

| No | Peneliti                                                                                                                    | Teknologi                                                   | Judul                                                                                                    | Fitur                                                                                                                                             |
|----|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Fajarini, Sri Dwi;<br>Kurniawati,<br>Juliana; Yuliani,<br>Fitria (2025)                                                     | NLP, Machine Learning (SVM, Random Forest, VADER)           | <i>Social Media Sentiment Analysis as a New Tool for Predicting Market Trends and Consumer Behaviour</i> | Analisis sentimen media sosial untuk mengidentifikasi pola opini publik dan memprediksi perilaku konsumen serta tren pasar.                       |
| 2  | Shrutika Rathore;<br>Rahul Nawkhare;<br>Navin Sharma;<br>Nitin Chaudhary;<br>Saurabh Chakole;<br>Bhaskar Vishwakrama (2025) | Dashboard analitik, visualisasi data, business intelligence | <i>Effective Data Visualization Techniques for Business Decision-Makers</i>                              | Penyajian data bisnis melalui dashboard analitik untuk meningkatkan efisiensi pengambilan keputusan dan perencanaan strategis.                    |
| 3  | Micheal, Dave (2024)                                                                                                        | React, TanStack Query, lazy loading, caching                | <i>React Query and Lazy Loading: Performance Optimization Best Practices</i>                             | Optimasi performa aplikasi <i>frontend</i> melalui pengelolaan data asinkron, caching, dan lazy loading untuk mengurangi permintaan API berulang. |

Berdasarkan kajian terhadap jurnal-jurnal penelitian terdahulu, dapat diidentifikasi adanya celah gap penelitian yang berkaitan dengan pemanfaatan hasil analisis sentimen media sosial pada sisi *frontend* aplikasi dashboard analitik. Sejumlah penelitian berfokus pada penerapan analisis sentimen menggunakan pendekatan Natural Language Processing (NLP) dan machine learning untuk memprediksi perilaku konsumen serta tren pasar. Namun demikian, penelitian-penelitian



tersebut umumnya menempatkan analisis sentimen sebagai fokus utama dan belum membahas secara mendalam bagaimana hasil analisis sentimen tersebut dikelola dan disajikan pada sisi *frontend*, khususnya dalam bentuk dashboard analitik yang digunakan secara langsung oleh pengguna.

Selain itu, penelitian lain menekankan pada peran dashboard dan teknik visualisasi data dalam meningkatkan efektivitas pengambilan keputusan bisnis. Fokus kajian lebih diarahkan pada desain visualisasi, jenis grafik, serta manfaat dashboard sebagai alat bantu analisis. Akan tetapi, aspek teknis pengelolaan data pada sisi *frontend*, seperti arsitektur pengambilan data dari API, pengelolaan server state, mekanisme caching, serta konsistensi data antar-komponen pada aplikasi *frontend* modern, masih belum menjadi perhatian utama dalam penelitian-penelitian tersebut.

Di sisi lain, terdapat penelitian yang membahas penggunaan React Query dalam pengelolaan data asinkron pada aplikasi React, termasuk pemanfaatan caching dan lazy loading untuk meningkatkan performa aplikasi *frontend*. Meskipun penelitian ini menunjukkan efektivitas React Query dalam mengurangi pemanggilan API berulang, konteks penerapannya masih bersifat umum dan belum secara spesifik dikaji sebagai bagian dari arsitektur *Client Data Layer* pada aplikasi dashboard analisis sentimen. Selain itu, keterkaitan antara pengelolaan server state di sisi *frontend* dengan kebutuhan penyajian data analitik pada konteks UMKM juga belum banyak dibahas. Oleh karena itu, penelitian ini mengisi celah tersebut dengan mengkaji penerapan arsitektur *Client Data Layer* menggunakan TanStack Query pada pengembangan Dashboard Analisis Sentimen UMKM.

## BAB 3

### METODOLOGI PENELITIAN

#### 3.1 Waktu dan Jadwal penelitian

##### 3.1.1 Waktu Pelaksanaan Penelitian

Waktu pelaksanaan penelitian ini direncanakan selama 6 bulan, yaitu dimulai pada bulan Agustus 2025 dan berakhir pada bulan Januari 2026. Rentang waktu tersebut dipilih untuk memastikan seluruh tahapan penelitian dapat dilaksanakan secara sistematis dan terstruktur, mulai dari analisis kebutuhan, perancangan sistem, implementasi, hingga pengujian. Pembagian waktu penelitian disusun secara bertahap agar setiap aktivitas penelitian dapat dilakukan secara optimal sesuai dengan metode yang digunakan, serta memberikan ruang untuk penyesuaian apabila ditemukan kendala selama proses pengembangan sistem.

##### 3.1.2 Jadwal Kegiatan Penelitian

Berikut adalah serangkaian jadwal kegiatan yang dilakukan selama pelaksanaan penelitian ini, yang diuraikan pada Tabel 3.1.

**Tabel 3.1** Jadwal Pelaksanaan Penelitian

| No | Nama Kegiatan                             | Waktu Pelaksanaan |     |     |     |     |      |
|----|-------------------------------------------|-------------------|-----|-----|-----|-----|------|
|    |                                           | 2025              |     |     |     |     | 2026 |
|    |                                           | Ags               | Sep | Okt | Nov | Des | Jan  |
| 1  | Analisis Kebutuhan Sistem                 |                   |     |     |     |     |      |
| 2  | Perumusan Ide dan Studi Awal Permasalahan |                   |     |     |     |     |      |
| 3  | Penyusunan Proposal dan Konsep Solusi     |                   |     |     |     |     |      |

| No | Nama Kegiatan                                                        | Waktu Pelaksanaan |     |     |     |     |      |
|----|----------------------------------------------------------------------|-------------------|-----|-----|-----|-----|------|
|    |                                                                      | 2025              |     |     |     |     | 2026 |
|    |                                                                      | Ags               | Sep | Okt | Nov | Des | Jan  |
| 4  | Pengumpulan Proposal dan Video Presentasi                            |                   |     |     |     |     |      |
| 5  | Seleksi Proposal dan Penetapan Finalis                               |                   |     |     |     |     |      |
| 6  | Presentasi dan Penjurian Akhir                                       |                   |     |     |     |     |      |
| 7  | Pengembangan Arsitektur <i>Frontend</i> dan <i>Client Data Layer</i> |                   |     |     |     |     |      |
| 8  | Implementasi Dashboard dan TanStack Query                            |                   |     |     |     |     |      |
| 9  | Pengujian Sistem (Black-box Testing)                                 |                   |     |     |     |     |      |
| 10 | Penyusunan Laporan Tugas Akhir                                       |                   |     |     |     |     |      |

Berdasarkan jadwal kegiatan penelitian yang telah disusun, pelaksanaan penelitian diawali pada bulan Agustus 2025 dengan tahap analisis kebutuhan sistem. Pada tahap ini dilakukan identifikasi permasalahan, pengumpulan kebutuhan pengguna, serta menyusun kebutuhan terhadap data dan API yang digunakan sebagai sumber data dashboard analisis sentimen. Tahap analisis ini menjadi dasar bagi tahapan penelitian selanjutnya. Pada bulan September 2025, kegiatan penelitian difokuskan pada perancangan sistem, yang meliputi perancangan arsitektur *Frontend*, perancangan *Client Data Layer*, serta perancangan antarmuka pengguna (UI). Tahap

perancangan bertujuan untuk menghasilkan rancangan sistem yang terstruktur dan sesuai dengan kebutuhan yang telah dianalisis sebelumnya. Pada periode bulan September hingga Desember 2025, kegiatan penelitian difokuskan pada tahapan perancangan dan implementasi sistem yang dilakukan secara iteratif dan saling tumpang tindih. Pada periode ini, proses perancangan antarmuka pengguna (desain antarmuka), perancangan sistem, perancangan arsitektur *Frontend* dan *Client Data Layer*, serta implementasi *Frontend* menggunakan TanStack Query tidak dilakukan secara terpisah dan linier, melainkan berjalan secara bersamaan sesuai dengan karakteristik metode Fountain yang fleksibel. Pendekatan ini memungkinkan hasil dari tahap implementasi *Frontend* untuk secara langsung digunakan sebagai dasar penyesuaian pada tahap perancangan sistem maupun perancangan arsitektur *Frontend*. Sebaliknya, perubahan pada desain antarmuka atau struktur arsitektur juga dapat segera diimplementasikan dan diuji tanpa harus menunggu selesainya seluruh tahapan sebelumnya. Dengan demikian, proses pengembangan sistem dapat berjalan lebih adaptif terhadap kebutuhan dan temuan selama penelitian. Pelaksanaan tahapan-tahapan tersebut secara paralel bertujuan untuk menjaga konsistensi antara rancangan arsitektur, mekanisme pengelolaan data pada *Client Data Layer*, serta implementasi antarmuka pengguna. Pola kerja ini sejalan dengan prinsip metode Fountain yang memungkinkan terjadinya pengulangan dan penyesuaian antar-tahapan pengembangan tanpa mengganggu keseluruhan alur penelitian. Selain itu, pendekatan paralel ini memberikan ruang bagi peneliti untuk melakukan evaluasi teknis secara berkelanjutan terhadap mekanisme pengelolaan data *Frontend*, khususnya terkait konsistensi data dan pengendalian pemanggilan API. Dengan adanya evaluasi berulang tersebut, setiap penyesuaian yang dilakukan dapat langsung divalidasi kesesuaiannya terhadap tujuan penelitian.

### **3.2 Metode Fountain**

Berdasarkan landasan teori dan pembahasan metode penelitian yang telah diuraikan pada Bab II, penelitian ini menggunakan metode Fountain sebagai pendekatan dalam pengembangan sistem. Metode Fountain dipilih karena memiliki karakteristik fleksibel dan iteratif, sehingga memungkinkan tahapan analisis, perancangan, implementasi, dan pengujian dilakukan secara saling tumpang tindih sesuai dengan kebutuhan penelitian. Karakteristik tersebut dinilai sesuai dengan pengembangan aplikasi *frontend* yang bersifat data-driven dan memerlukan evaluasi berulang terhadap arsitektur dan pengelolaan data. Pada Bab III ini, metode Fountain diterapkan secara sistematis pada penelitian yang dilakukan. Pembahasan difokuskan pada tahapan penerapan metode Fountain dalam konteks pengembangan Dashboard Analisis Sentimen, mulai dari tahap analisis hingga tahap testing. Uraian pada setiap tahapan menjelaskan aktivitas yang dilakukan dalam penelitian ini tanpa mengulang pembahasan teoritis yang telah

disampaikan pada Bab II. Pemilihan metode Software Development Life Cycle (SDLC) yang tepat merupakan langkah strategis yang krusial untuk memastikan keberhasilan proyek serta mencegah pembengkakan biaya dan waktu pengembangan (Aniley et al., 2024). Berdasarkan pertimbangan tersebut, penelitian ini mengadopsi metode Fountain yang dinilai relevan karena memiliki karakteristik fleksibel namun tetap terstruktur. Efektivitas metode ini didukung oleh penelitian (Sastra and Sutawinata, 2023) pada perancangan sistem SIP-PTK, yang menunjukkan bahwa model Fountain mampu memandu tahapan analisis, desain, implementasi, hingga pengujian secara efektif pada sistem yang memiliki kebutuhan pengolahan data yang spesifik. Relevansi metode Fountain juga sejalan dengan fokus penelitian ini, yaitu pengembangan aplikasi *frontend* yang bersifat data-driven dan memiliki ketergantungan tinggi terhadap interaksi antar-komponen. Dalam pengembangan *frontend*, tahapan perancangan arsitektur, implementasi, dan pengujian tidak berjalan secara linier, melainkan saling berkaitan dan sering memerlukan penyesuaian berdasarkan hasil evaluasi sistem. Penerapan metode Fountain memungkinkan tahapan pengembangan dan testing berjalan secara paralel serta saling tumpang tindih (*overlapping*) tanpa menghilangkan struktur dan urutan penelitian yang jelas. Melalui pendekatan ini, aktivitas analisis kebutuhan, perancangan arsitektur *frontend*, implementasi *Client Data Layer*, serta pengujian sistem dapat dilakukan secara berulang dan saling mempengaruhi. Hasil pada satu tahapan dapat langsung digunakan sebagai dasar penyesuaian pada tahapan lainnya, seperti perubahan struktur data atau mekanisme sinkronisasi, tanpa harus menunggu seluruh siklus pengembangan selesai. Dengan demikian, proses pengembangan sistem menjadi lebih adaptif dan terkontrol, sehingga diharapkan mampu menghasilkan hasil penelitian yang sesuai dengan tujuan penelitian secara optimal. Tahapan penerapan metode fountain dalam penelitian ini terdiri atas beberapa tahap sebagai berikut:

### **3.2.1 Analysis**

Pada tahap analisis, penelitian ini mengkaji kebutuhan dan permasalahan pada pengembangan aplikasi *frontend* berbasis web yang berfungsi sebagai dashboard analisis sentimen UMKM. Analisis dilakukan terhadap kebutuhan penyajian hasil analisis sentimen media sosial dalam bentuk visualisasi yang informatif, ringkas, dan mudah dipahami oleh pengguna. Informasi yang dianalisis mencakup kebutuhan penampilan ringkasan sentimen, distribusi sentimen, serta indikator lain yang relevan untuk memantau persepsi konsumen secara umum. Analisis juga difokuskan pada alur pengelolaan data antara backend dan *frontend*. Pada penelitian ini, proses pengumpulan data media sosial dan analisis sentimen sepenuhnya dilakukan di sisi backend, sementara *frontend* bertugas mengonsumsi data hasil analisis melalui REST API. Kondisi ini menuntut adanya mekanisme pengelolaan data *frontend* yang mampu

menerima data secara konsisten dan menyajikannya ke berbagai komponen dashboard tanpa menimbulkan inkonsistensi informasi. Selain itu, pada tahap analisis diidentifikasi permasalahan pengelolaan data *frontend* pada aplikasi dashboard yang bersifat data-driven, khususnya ketika data yang sama digunakan oleh banyak komponen antarmuka secara bersamaan. Permasalahan yang dianalisis meliputi potensi terjadinya permintaan data berulang, kesulitan sinkronisasi data antar-komponen, serta kebutuhan pembaruan data secara terkontrol. Hasil analisis ini menjadi dasar dalam menentukan kebutuhan arsitektur *frontend* yang lebih terstruktur dan efisien. Pengguna sistem dalam penelitian ini dianalisis sebagai pengguna umum atau pelaku UMKM yang memanfaatkan dashboard untuk memantau informasi sentimen. Aktivitas pengguna dibatasi pada pengamatan dan eksplorasi informasi yang ditampilkan, tanpa keterlibatan langsung dalam proses pengolahan data. Dengan karakteristik pengguna tersebut, analisis sistem difokuskan pada aspek penyajian informasi dan pengelolaan data di sisi *frontend* agar sesuai dengan tujuan penelitian.

### **3.2.2 Requirement Specification**

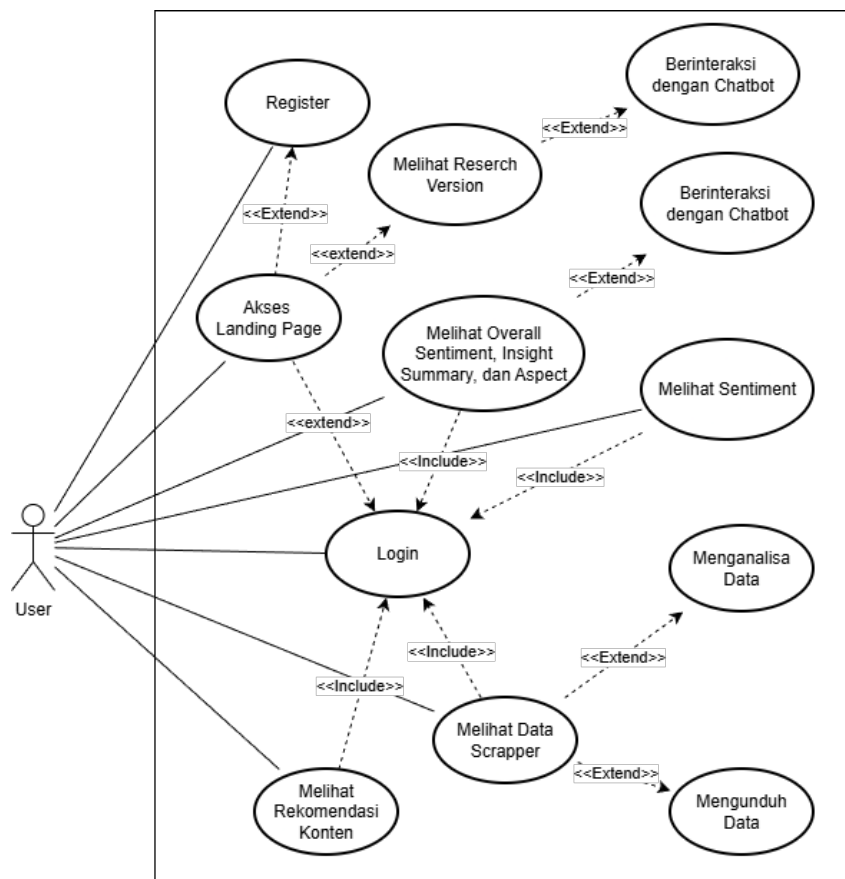
Tahap *requirement specification* bertujuan untuk merumuskan kebutuhan sistem secara terstruktur berdasarkan hasil analisis yang telah dilakukan pada tahap sebelumnya. Pada penelitian ini, spesifikasi kebutuhan difokuskan pada sisi *frontend* dashboard analisis sentimen UMKM, dengan mempertimbangkan karakteristik sistem yang bersifat data-driven dan bergantung pada data hasil analisis sentimen dari backend. Kebutuhan fungsional sistem mencakup kemampuan aplikasi *frontend* untuk mengonsumsi data hasil analisis sentimen yang disediakan melalui REST API dan menyajikannya dalam berbagai komponen dashboard. Sistem harus mampu menampilkan ringkasan sentimen, distribusi sentimen, serta indikator pendukung lainnya secara konsisten pada seluruh komponen antarmuka. Selain itu, sistem perlu mendukung pembaruan data berdasarkan hasil analisis sentimen yang dipilih pengguna, sehingga informasi yang ditampilkan tetap relevan dengan dataset terbaru tanpa memerlukan pengambilan data ulang secara manual pada setiap komponen antarmuka. Selain kebutuhan fungsional, sistem juga memiliki kebutuhan non-fungsional yang berkaitan dengan kualitas pengelolaan data dan performa aplikasi *frontend*. Kebutuhan non-fungsional tersebut meliputi konsistensi data antar-komponen antarmuka, efisiensi dalam pengambilan data dari API, serta mekanisme pengelolaan cache untuk mengurangi permintaan data yang tidak diperlukan. Sistem diharapkan mampu mengelola data secara terpusat di sisi *frontend* sehingga setiap komponen menggunakan sumber data yang sama dan tersinkronisasi. Kebutuhan lain yang menjadi perhatian pada tahap ini adalah kebutuhan kemudahan pengembangan dan pemeliharaan sistem. Struktur pengelolaan data *frontend* harus memungkinkan penambahan atau perubahan komponen dashboard tanpa

memengaruhi keseluruhan sistem secara signifikan. Dengan demikian, spesifikasi kebutuhan ini menjadi dasar dalam perancangan arsitektur *frontend* dan penerapan *Client Data Layer* pada tahap desain dan implementasi selanjutnya.

### 3.2.3 Design

Tahap *design* merupakan tahapan perancangan solusi berdasarkan spesifikasi kebutuhan sistem yang telah dirumuskan pada tahap *requirement specification*. Pada tahap ini, kebutuhan sistem yang bersifat konseptual diterjemahkan ke dalam rancangan teknis yang menjadi acuan dalam proses implementasi *frontend* dashboard analisis sentimen. Perancangan difokuskan pada bagaimana sistem *frontend* mengelola dan menyajikan data secara terstruktur, konsisten, dan mudah dikembangkan sesuai dengan tujuan penelitian. Pada penelitian ini, tahap *design* mencakup beberapa aspek utama, yaitu *use case diagram*, *entity relationship diagram*, alur sistem keseluruhan, perancangan arsitektur *frontend*, perancangan *Client Data Layer* sebagai mekanisme pengelolaan data dari API, serta perancangan antarmuka pengguna dalam bentuk *wireframe*. Setiap aspek perancangan memiliki peran yang saling berkaitan dalam membangun sistem *frontend* yang bersifat *data-driven*.

#### 1. Use Case Diagram



**Gambar 3.1** Use Case Diagram

Use Case Diagram pada Gambar 3.1 menggambarkan interaksi utama antara aktor dan sistem Dashboard Analisis Sentimen yang dikembangkan dalam penelitian ini. Aktor utama pada sistem adalah User, yang merepresentasikan pengguna akhir seperti pelaku UMKM atau pemilik bisnis yang memanfaatkan sistem untuk memperoleh informasi analisis sentimen dari data media sosial. Interaksi pengguna dengan sistem diawali melalui use case Akses Landing Page, yang dapat diakses tanpa proses autentikasi. Dari use case ini, pengguna memiliki opsi untuk melakukan Register sebagai pengguna baru atau Login bagi pengguna yang telah terdaftar, yang dimodelkan menggunakan relasi extend karena kedua proses tersebut bersifat opsional dan bergantung pada kondisi pengguna.

Setelah berhasil melakukan Login, pengguna dapat mengakses fitur-fitur utama sistem. Salah satu fitur utama adalah Melihat Sentiment, yang digunakan untuk menampilkan hasil analisis sentimen dari data yang telah diproses. Use case Melihat Sentiment memiliki relasi include dengan Menganalisa Data, yang menunjukkan bahwa proses analisis data merupakan bagian yang tidak terpisahkan dari penyajian informasi sentimen. Hasil analisis tersebut kemudian dapat diperluas melalui relasi extend ke use case Melihat Overall Sentiment, Insight Summary, dan Aspect Summary, yang memungkinkan pengguna memperoleh ringkasan sentimen secara keseluruhan, wawasan utama, serta analisis sentimen berbasis aspek.

Selain itu, sistem juga menyediakan fitur Melihat Data Scrapper yang memungkinkan pengguna melihat data hasil proses pengambilan data dari media sosial. Dari use case ini, terdapat relasi extend ke Mengunduh Data, yang menunjukkan bahwa proses pengunduhan bersifat opsional dan hanya dilakukan apabila pengguna membutuhkan data tersebut dalam bentuk berkas. Di sisi lain, pengguna juga dapat mengakses fitur Melihat Rekomendasi Konten, yang disediakan sebagai bagian dari layanan sistem untuk memberikan rekomendasi berdasarkan hasil analisis sentimen. Keseluruhan relasi use case ini menggambarkan alur penggunaan sistem secara terstruktur, mulai dari akses awal hingga pemanfaatan fitur analisis dan penyajian informasi sentimen.



## 2. Entity Relationship Diagram



**Gambar 3.2** Entity Relationship Diagram

ERD pada Gambar 3.2 menggambarkan bagaimana data hasil pengambilan data media sosial, hasil analisis sentimen, serta rekomendasi konten disimpan dan saling terhubung dalam basis data.

Entitas *users* berperan sebagai entitas utama yang menyimpan informasi pengguna sistem, termasuk identitas dan data autentikasi. Setiap pengguna dapat menghasilkan satu atau lebih data hasil pengambilan media sosial yang direpresentasikan oleh entitas *scrape\_results*, yang menyimpan informasi profil serta data mentah hasil proses scraping. Relasi antara entitas *users* dan *scrape\_results* menunjukkan hubungan satu-ke-banyak, di mana satu pengguna dapat memiliki beberapa hasil pengambilan data.

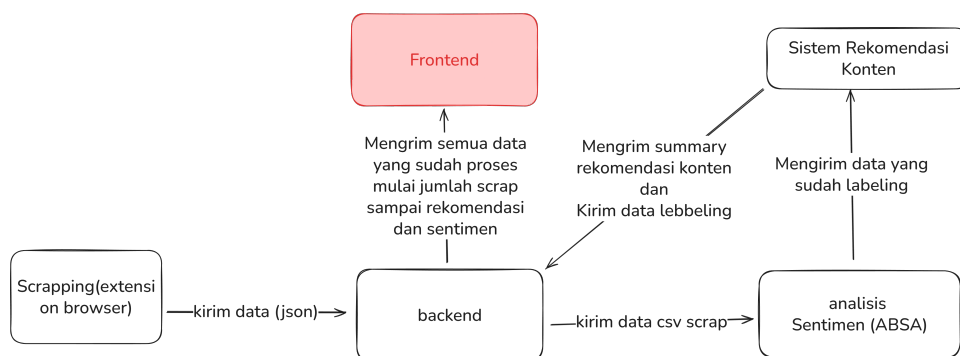
Hasil proses pengambilan data selanjutnya dianalisis dan disimpan pada entitas *sentiment\_result*, yang memiliki relasi satu-ke-satu atau satu-ke-banyak dengan *scrape\_results* tergantung pada kebutuhan analisis. Untuk mendukung analisis sentimen berbasis aspek, entitas *sentiment\_comments* digunakan untuk menyimpan komentar beserta kategori aspek sentimen seperti *food\_quality*, *price*, dan

service. Entitas ini memiliki relasi satu-ke-banyak dengan `sentiment_result`, yang memungkinkan satu hasil analisis sentimen memiliki banyak komentar terkait.

Selain analisis sentimen, sistem juga menyediakan fitur rekomendasi konten yang dimodelkan melalui entitas `recommendation_result`. Entitas ini berelasi dengan `sentiment_result` sebagai dasar penyusunan rekomendasi. Detail rekomendasi disimpan pada beberapa entitas turunan, yaitu `recommendation_hashtags`, `recommendation_captions`, dan `recommendation_best_posting`, yang masing-masing memiliki relasi satu-ke-banyak terhadap `recommendation_result`. Struktur ini memungkinkan sistem menyimpan berbagai bentuk rekomendasi secara terpisah namun tetap terhubung pada satu konteks hasil analisis.

Selain itu, entitas `langchain_documents` digunakan untuk menyimpan data teks, metadata, dan embedding vektor yang mendukung proses analisis lanjutan atau pemrosesan berbasis kecerdasan buatan. `Langchain_documents` berfungsi sebagai penyimpanan dokumen pendukung di luar alur utama pengelolaan data dashboard.

### 3. Alur Sistem Keseluruhan



**Gambar 3.3** Alur Sistem Keseluruhan

Alur kerja keseluruhan sistem dimulai dari Dashboard, yang berfungsi sebagai pusat interaksi utama bagi pengguna. Dari dashboard, pengguna dapat mengakses halaman Scraper untuk melakukan proses pengambilan data media sosial. Pada tahap ini, pengguna memanfaatkan layanan scraper yang terintegrasi dengan extension scraper untuk mengumpulkan data dari media sosial. Setelah proses pengambilan data selesai, data hasil scraping dikirimkan ke backend sistem dan selanjutnya disimpan ke dalam *database* sebagai data mentah.

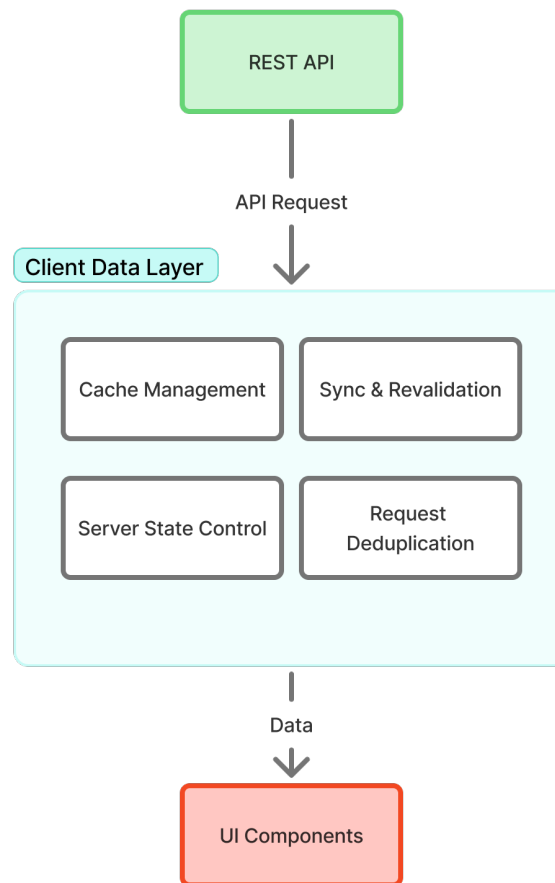
Data hasil scraping yang telah tersimpan kemudian dapat diproses lebih lanjut melalui tahap analisis data, di mana sistem melakukan analisis sentimen berbasis aspek

(Aspect-Based Sentiment Analysis/ABSA). Proses ABSA ini menghasilkan informasi sentimen yang terstruktur berdasarkan aspek-aspek tertentu. Hasil analisis sentimen tersebut selanjutnya diteruskan ke modul rekomendasi konten, yang berfungsi untuk menghasilkan rekomendasi konten berdasarkan pola dan hasil sentimen yang diperoleh.

Setelah proses rekomendasi selesai, data rekomendasi konten beserta hasil analisis sentimen dikirim kembali ke backend dan disimpan ke dalam *database*. Pada tahap akhir, sistem menyajikan kembali data hasil scraping, hasil analisis sentimen, serta rekomendasi konten ke Dashboard, sehingga pengguna dapat melihat, mengevaluasi, dan memanfaatkan seluruh informasi yang dihasilkan oleh sistem secara terintegrasi.

#### 4. Perancangan Arsitektur *Frontend*

Penelitian ini menerapkan prinsip *component-based architecture*, di mana antarmuka pengguna dibangun dari komponen-komponen modular yang memiliki tanggung jawab spesifik dan dapat digunakan kembali. Setiap komponen difokuskan pada penyajian data dan interaksi pengguna, sementara logika pengelolaan data dipisahkan ke dalam lapisan tersendiri agar struktur aplikasi lebih terorganisasi dan mudah dipelihara. Arsitektur *frontend* dirancang dengan pendekatan *data-driven*, di mana tampilan antarmuka sepenuhnya bergantung pada data yang dikelola oleh sistem. Untuk mendukung hal tersebut, prinsip *separation of concerns* diterapkan dengan memisahkan lapisan presentasi dan lapisan pengelolaan data, sehingga komponen antarmuka tidak berinteraksi langsung dengan REST API.



**Gambar 3.4** Diagram Arsitektur *Frontend*

Diagram arsitektur *frontend* pada Gambar 3.4 digunakan untuk menggambarkan pembagian lapisan sistem serta alur pengelolaan data pada sistem yang dikembangkan. Diagram ini menunjukkan bagaimana data dari REST API dikelola melalui *Client Data Layer* sebelum disajikan pada komponen antarmuka pengguna. Lapisan REST API diposisikan sebagai sumber data eksternal yang menyediakan data hasil analisis sentimen. Seluruh proses pengolahan data, termasuk pengambilan data media sosial dan analisis sentimen, dilakukan pada sisi backend dan berada di luar ruang lingkup penelitian ini. *Frontend* berperan sebagai konsumen data yang mengakses informasi tersebut melalui antarmuka REST API. Lapisan *Client Data Layer* berfungsi sebagai lapisan perantara antara REST API dan komponen antarmuka pengguna. Data yang diperoleh dari REST API dikelola dan dimodelkan secara terpusat sebelum disajikan pada komponen antarmuka pengguna. Lapisan ini bertanggung jawab dalam proses pengambilan data, penyimpanan sementara (*caching*), serta sinkronisasi data antar-komponen. Lapisan *UI Components* merupakan lapisan presentasi yang bertugas menampilkan data kepada pengguna dan menangani interaksi pengguna dengan sistem. Komponen pada lapisan ini menerima data yang telah dikelola oleh

*Client Data Layer* dan menyajikannya dalam bentuk visualisasi seperti grafik dan tabel.

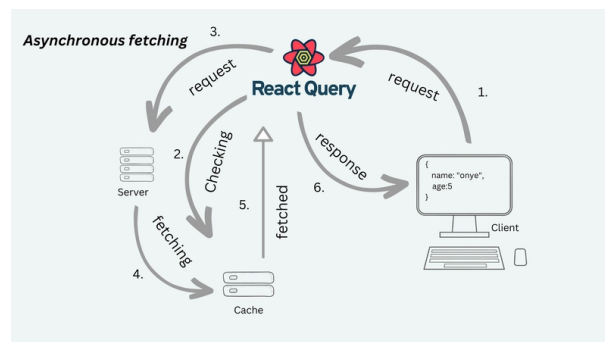
## 5. Perancangan *Client Data Layer*

Perancangan *Client Data Layer* dilakukan untuk mengelola data yang bersumber dari backend secara terpusat pada sisi *frontend* sebelum disajikan pada komponen antarmuka pengguna. Pada aplikasi dashboard yang bersifat data-driven, data yang sama dapat digunakan oleh berbagai komponen secara bersamaan dan diperbarui secara dinamis. Oleh karena itu, diperlukan suatu lapisan pengelolaan data yang mampu mengatur alur data, menjaga konsistensi informasi, serta mengendalikan interaksi antara *frontend* dan REST API. *Client Data Layer* diposisikan sebagai lapisan perantara antara REST API dan komponen antarmuka pengguna, sebagaimana ditunjukkan pada Gambar 3.4 diagram arsitektur *frontend*. Seluruh data yang diperoleh dari backend tidak langsung digunakan oleh komponen antarmuka, melainkan terlebih dahulu dikelola melalui *Client Data Layer*. Dengan pendekatan ini, komponen antarmuka tidak perlu mengetahui detail proses pengambilan data dari API, sehingga fokus komponen dapat diarahkan pada penyajian data dan interaksi pengguna. Secara konseptual, *Client Data Layer* memiliki beberapa tanggung jawab utama dalam sistem *frontend*. Tanggung jawab tersebut meliputi proses pengambilan data dari REST API, pengelolaan server state, penyimpanan sementara data melalui mekanisme caching, serta sinkronisasi data antar-komponen antarmuka. Dengan pengelolaan data yang terpusat, permintaan data yang bersifat berulang dapat dikendalikan dan setiap komponen antarmuka memperoleh data yang konsisten sesuai dengan kondisi sistem. Selain pengelolaan alur data, *Client Data Layer* juga dirancang untuk menangani pemodelan data sebelum digunakan oleh komponen antarmuka. Data yang diperoleh dari REST API dimodelkan secara terstruktur pada *Client Data Layer* agar memiliki bentuk dan konsistensi yang jelas. Pendekatan ini bertujuan untuk meminimalkan ketergantungan komponen antarmuka terhadap struktur data mentah dari backend serta mempermudah proses pengembangan dan pemeliharaan sistem *frontend*. Dalam penelitian ini, *Client Data Layer* dirancang untuk diimplementasikan menggunakan TanStack Query sebagai pustaka pengelolaan server state pada *frontend*. Pemilihan TanStack Query didasarkan pada kemampuannya dalam menyediakan mekanisme pengelolaan data asinkron secara terpusat, termasuk caching, sinkronisasi data, dan pengendalian permintaan data. Dengan memanfaatkan pustaka tersebut, *Client Data Layer* diharapkan mampu

mendukung pengelolaan data *frontend* yang lebih terstruktur, konsisten, dan efisien sesuai dengan kebutuhan dashboard analisis sentimen.

## 6. Perancangan Penggunaan TanStack Query

Perancangan penggunaan TanStack Query pada penelitian ini mengacu pada dokumentasi resmi TanStack Query sebagai pustaka server state management untuk aplikasi *frontend*. Berdasarkan dokumentasi resmi TanStack Query (Tanstack LCC, 2025), pustaka ini dirancang untuk mengelola data asinkron yang bersumber dari API secara terpusat melalui mekanisme pengambilan data, penyimpanan sementara (caching), serta sinkronisasi data antar-komponen antarmuka. Pendekatan tersebut memungkinkan komponen *frontend* memperoleh data yang konsisten tanpa harus melakukan permintaan data secara langsung ke REST API, sehingga pemisahan tanggung jawab antara lapisan presentasi dan lapisan pengelolaan data dapat terjaga. Dengan karakteristik tersebut, TanStack Query dinilai sesuai untuk diimplementasikan sebagai *Client Data Layer* pada aplikasi *frontend* yang bersifat data-driven, khususnya dalam konteks dashboard analisis sentimen yang membutuhkan konsistensi data dan pembaruan informasi secara terkontrol.



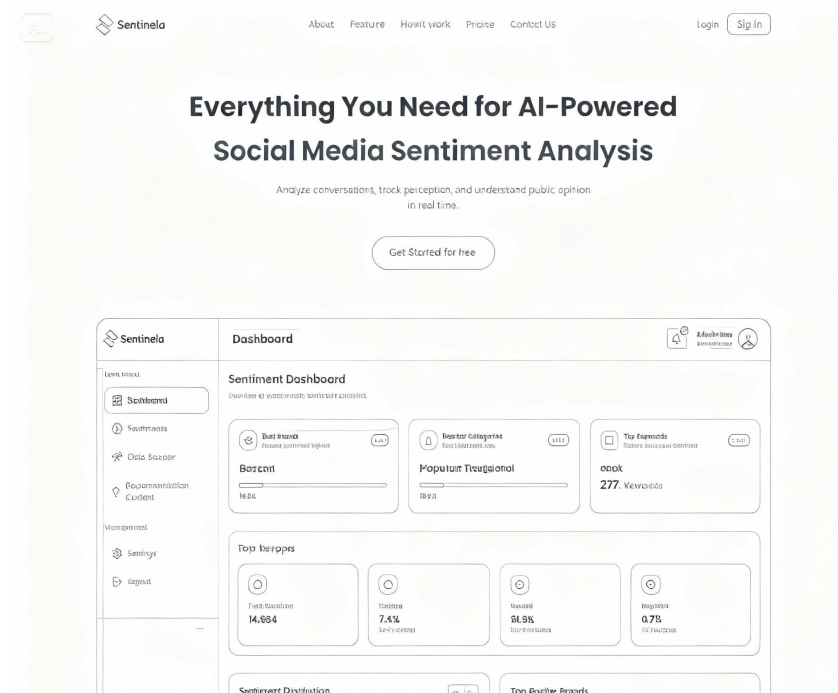
**Gambar 3.5** Cara Kerja TanStack Query

Gambar 3.5 menunjukkan alur pengelolaan data asinkron pada sisi *frontend* menggunakan TanStack Query. Ketika komponen antarmuka membutuhkan data, permintaan tidak langsung dikirimkan ke REST API, melainkan terlebih dahulu dikelola oleh *Client Data Layer*. TanStack Query melakukan pengecekan terhadap cache untuk menentukan apakah data yang diminta masih valid. Jika data tersedia dan masih relevan, data langsung dikembalikan ke komponen antarmuka tanpa melakukan pemanggilan ulang ke server. Sebaliknya, apabila data tidak tersedia atau sudah tidak valid, sistem akan melakukan proses fetching ke REST API dan menyimpan hasilnya ke dalam cache sebelum disajikan ke antarmuka pengguna. Mekanisme ini

memungkinkan pengelolaan data yang lebih efisien, mengurangi jumlah permintaan API yang tidak perlu, serta menjaga konsistensi data antar-komponen pada *frontend* dashboard analisis sentimen.

## 7. Perancangan Antarmuka

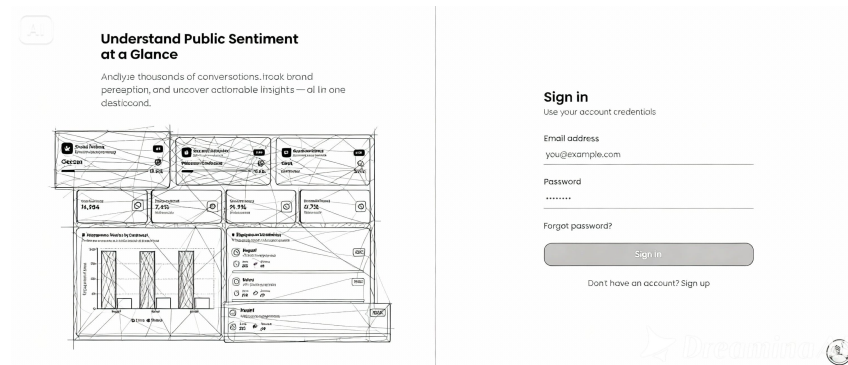
Rancangan antarmuka dilakukan sebagai tindak lanjut dari struktur arsitektur yang telah ditetapkan, dengan tujuan memastikan bahwa data hasil analisis sentimen yang dikelola oleh sistem dapat disajikan kepada pengguna secara informatif, mudah dipahami, dan konsisten. Antarmuka pengguna dirancang untuk merepresentasikan kebutuhan fungsional sistem dalam bentuk visual, sekaligus menjadi media interaksi antara pengguna dan sistem *frontend* dashboard analisis sentimen. Sebagai bentuk konkret dari perancangan antarmuka pengguna, dilakukan penyusunan wireframe yang menggambarkan tata letak komponen, alur navigasi, serta penyajian informasi pada setiap halaman utama sistem.



**Gambar 3.6** Wireframe Landing Page

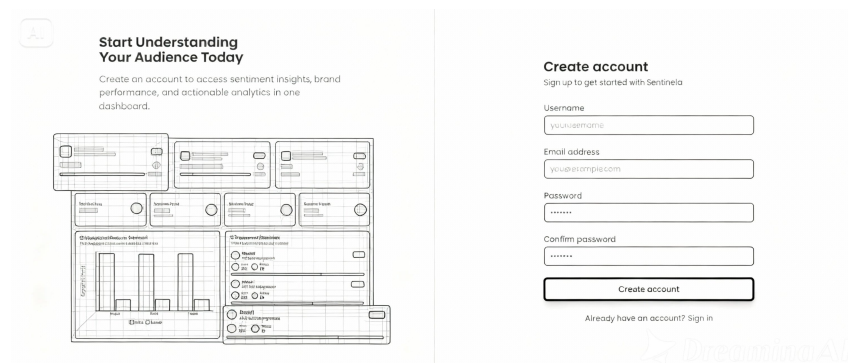
Landing Page pada Gambar 3.6 dirancang sebagai halaman awal yang pertama kali diakses oleh pengguna ketika membuka sistem. Halaman ini berfungsi untuk memberikan gambaran umum mengenai tujuan dan fitur utama sistem sebelum pengguna melakukan proses autentikasi. Struktur halaman dirancang sederhana dengan penekanan pada informasi pengenalan sistem serta elemen navigasi utama yang mengarahkan pengguna ke halaman login atau registrasi. Perancangan

wireframe ini bertujuan untuk memastikan pengguna dapat memahami konteks sistem secara cepat dan memiliki alur navigasi yang jelas menuju fitur utama yang disediakan.



**Gambar 3.7** Wireframe Halaman Login

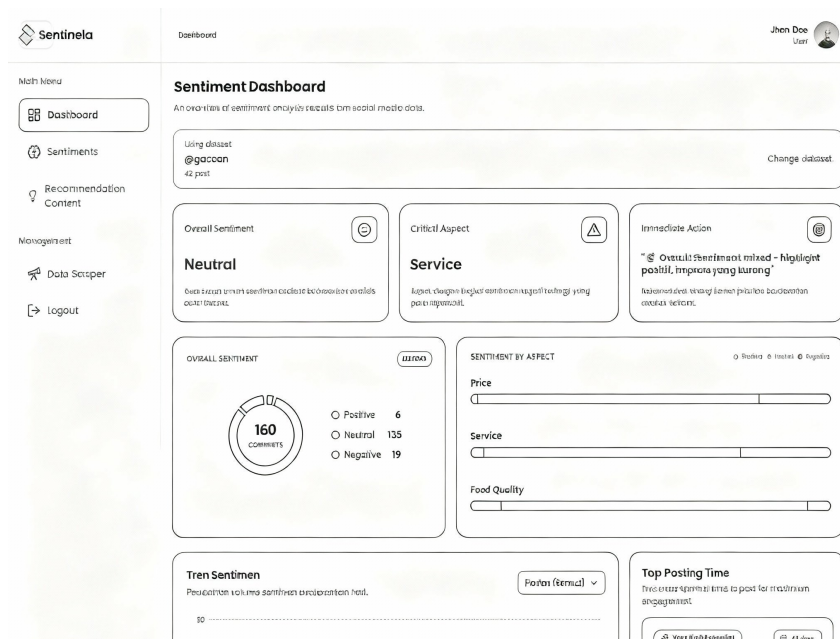
Halaman Login pada Gambar 3.7 dirancang sebagai antarmuka autentikasi pengguna untuk mengakses fitur utama sistem. Halaman ini menyediakan elemen input untuk memasukkan kredensial pengguna untuk memproses autentikasi. pengguna dapat melakukan proses login sebelum diarahkan ke halaman dashboard. Halaman ini dirancang untuk mendukung keamanan akses sistem dengan memastikan bahwa hanya pengguna yang telah terautentikasi yang dapat mengakses fitur-fitur utama aplikasi.



**Gambar 3.8** Wireframe Halaman Register

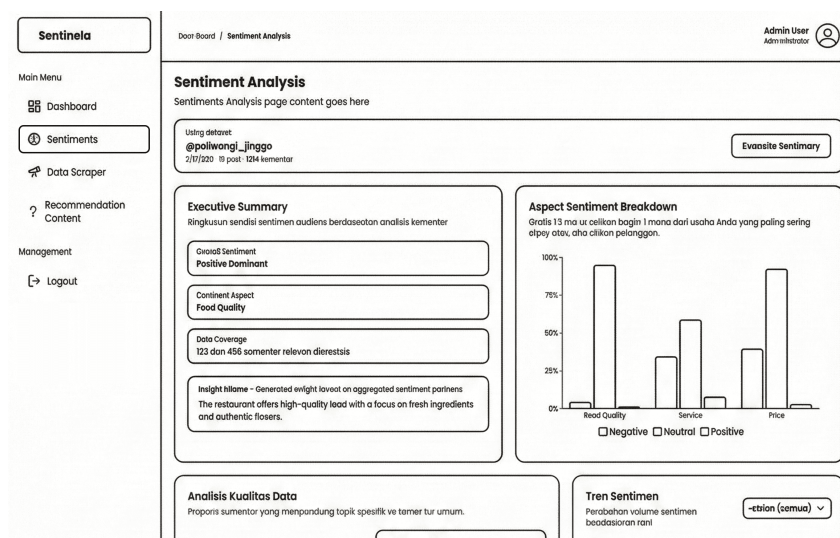
Halaman Register pada Gambar 3.8 dirancang sebagai antarmuka pendaftaran pengguna baru sebelum dapat mengakses sistem. Halaman ini menyediakan form untuk pengisian data pengguna yang diperlukan dalam proses registrasi. Pengguna dapat melakukan proses pendaftaran secara sistematis. Halaman ini juga berfungsi sebagai bagian dari mekanisme kontrol akses dengan memastikan bahwa data pengguna dikumpulkan dan diproses sebelum akun dapat digunakan untuk mengakses fitur utama sistem.





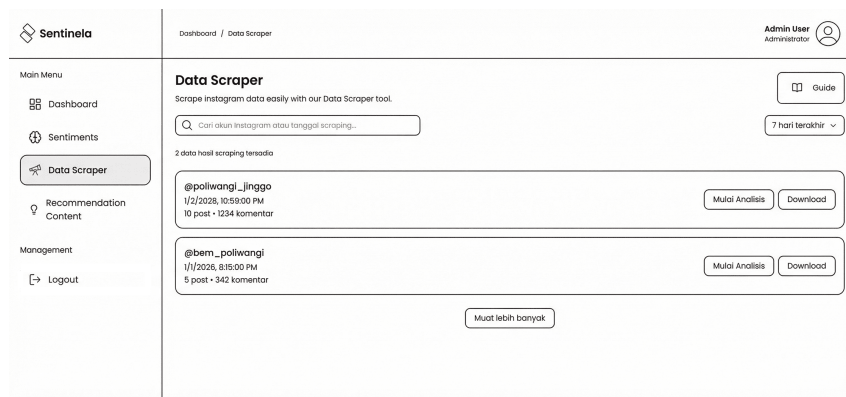
**Gambar 3.9** Wireframe Halaman Dashboard

Halaman Dashboard pada Gambar 3.9 dirancang sebagai halaman utama setelah pengguna berhasil melakukan autentikasi. Halaman ini berfungsi sebagai pusat informasi yang menampilkan ringkasan data dan visualisasi utama dari sistem. Struktur dashboard dirancang untuk memudahkan pengguna dalam memantau kondisi data secara keseluruhan serta mengakses fitur-fitur utama melalui navigasi yang tersedia. Perancangan wireframe ini bertujuan untuk memastikan penyajian informasi yang terstruktur dan mudah dipahami, sehingga pengguna dapat memperoleh gambaran umum hasil analisis secara cepat sebelum melakukan eksplorasi data lebih lanjut.



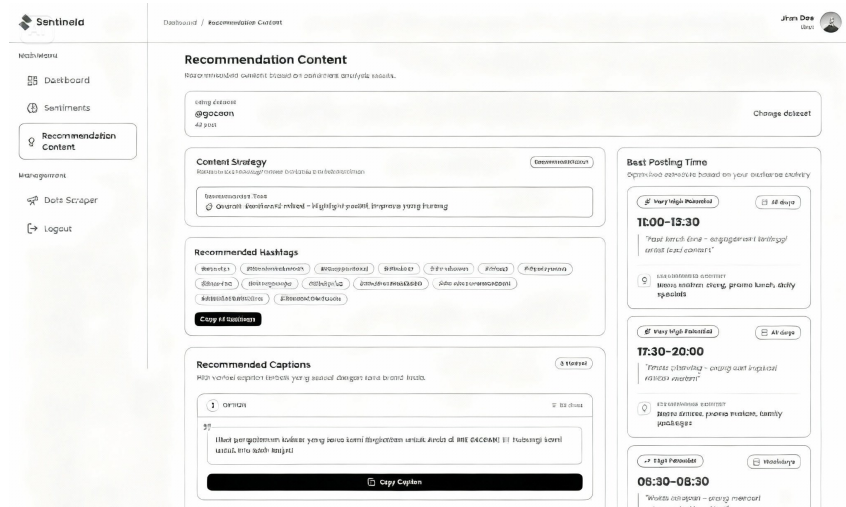
**Gambar 3.10** Wireframe Halaman Sentiment

Halaman Sentiment pada Gambar 3.10 dirancang untuk menampilkan hasil analisis sentimen aspect based sentiment analysis secara lebih rinci dibandingkan halaman dashboard. Halaman ini menyajikan informasi sentimen dalam bentuk visualisasi data yang memudahkan pengguna dalam memahami distribusi dan kecenderungan sentimen. Perancangan struktur halaman difokuskan pada penyajian data yang terorganisasi dan mudah diinterpretasikan, sehingga pengguna dapat melakukan analisis sentimen secara lebih mendalam sesuai dengan kebutuhan informasi yang diinginkan.



**Gambar 3.11** Wireframe Halaman Scraper

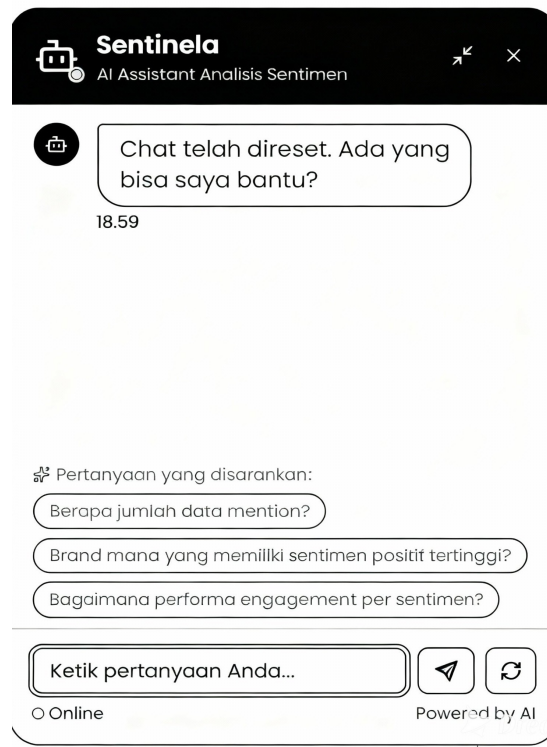
Halaman Scraper pada Gambar 3.11 dirancang sebagai antarmuka yang memfasilitasi proses pengumpulan dan pemantauan data yang bersumber dari media sosial. Halaman ini menyajikan data hasil proses scraping yang dilakukan pada sisi server, sehingga pengguna dapat mengetahui data yang tersedia dan data yang mana yang akan digunakan oleh sistem pada tahap analisis.



**Gambar 3.12** Wireframe Halaman Recommendation

Halaman Recommendation pada Gambar 3.12 dirancang sebagai antarmuka yang

menyajikan rekomendasi berdasarkan hasil analisis sentimen yang telah diproses secara otomatis oleh sistem. Halaman ini menampilkan informasi rekomendasi yang diperoleh dari pengolahan data sentimen, sehingga pengguna dapat memahami insight yang dihasilkan dari data media sosial. Hasil rekomendasi yang ditampilkan pada halaman ini juga disajikan secara lebih detail sebagai bagian dari informasi rekomendasi utama. Halaman ini bertujuan untuk memudahkan pengguna dalam mengakses rekomendasi dan memahami hasil analisis sentimen secara terstruktur.



**Gambar 3.13** Wireframe Halaman Chatbot

Halaman Chatbot pada Gambar 3.13 dirancang sebagai antarmuka yang memfasilitasi interaksi tanya jawab antara pengguna dengan sistem. Halaman ini memungkinkan pengguna untuk mengajukan pertanyaan terkait data yang tersedia dan memperoleh jawaban yang relevan berdasarkan informasi yang telah diproses oleh sistem. Halaman ini bertujuan untuk memudahkan pengguna dalam mengakses informasi dan memperoleh insight yang dibutuhkan secara interaktif.

#### **3.2.4 Coding (Implementation)**

Tahap coding diawali dengan penyiapan lingkungan pengembangan *frontend* dan penyesuaian struktur proyek agar selaras dengan arsitektur yang telah dirancang. Framework React dipilih sebagai fondasi antarmuka pengguna, sedangkan TanStack Query diterapkan sebagai mekanisme utama pengelolaan server state yang membentuk *Client Data Layer*.

Dalam mengimplementasikan struktur direktori dan manajemen pengambilan data (data *fetching*), penelitian ini mengadopsi pendekatan arsitektur berlapis (*layered architecture*) yang direkomendasikan dalam praktik produksi React modern (Owusu, 2025). Sesuai pola tersebut, implementasi dilakukan dengan memisahkan logika API ke dalam service layer, yang kemudian dibungkus menggunakan custom hooks berbasis TanStack Query. Pendekatan ini memastikan bahwa komponen UI hanya berinteraksi dengan hooks tanpa perlu mengetahui kompleksitas implementasi REST API secara langsung.

Bersamaan dengan itu, dilakukan pula penyiapan komponen pendukung dan integrasi data dengan backend. Seluruh persiapan ini bertujuan untuk menjamin bahwa proses implementasi berjalan secara terstruktur, menjaga konsistensi kode, serta memenuhi spesifikasi kebutuhan sistem yang telah didefinisikan pada tahap sebelumnya.

### **3.2.5 Testing**

Metode pengujian yang digunakan dalam penelitian ini mengacu pada konsep black-box testing, dengan pendekatan scenario-based testing. Pengujian dilakukan dengan mengamati perilaku sistem berdasarkan skenario penggunaan tanpa memperhatikan struktur internal kode program. Pendekatan ini menempatkan sistem sebagai sebuah kesatuan yang diuji dari sudut pandang pengguna, sehingga pengujian difokuskan pada kesesuaian fungsi dan respons sistem terhadap alur penggunaan yang dirancang. Pemilihan pendekatan scenario-based testing didasarkan pada karakteristik sistem yang dikembangkan, yaitu dashboard analitik yang bersifat data-driven dan mengandalkan interaksi antar-komponen antarmuka. Oleh karena itu, pengujian diarahkan pada pengetesan perilaku sistem dalam menampilkan data, menjaga konsistensi informasi, serta merespons pembaruan data sesuai dengan kondisi yang terjadi. Ruang lingkup pengujian pada penelitian ini difokuskan pada perilaku sistem *frontend*, khususnya pada pengelolaan data melalui *Client Data Layer* dan penyajian data pada antarmuka pengguna. Pengujian tidak mencakup pengetesan terhadap algoritma analisis sentimen maupun proses pengolahan data pada sisi backend. Skenario pengujian disusun berdasarkan fitur utama sistem dan merepresentasikan alur penggunaan dari sudut pandang pengguna. Setiap skenario dirancang untuk mengevaluasi perilaku sistem *frontend* dalam merespons interaksi pengguna serta memastikan kesesuaian fungsi sistem dengan rancangan yang telah ditetapkan. Skenario pengujian dalam penelitian ini terdiri atas beberapa pengujian sebagai berikut:

## 1. Landing Page

**Tabel 3.2** Skenario Pengujian Landing Page

| ID       | Test Step                                                                                             | Skenario Pengujian                                      | Data Uji | Expected Result                                                                                         |
|----------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------|----------|---------------------------------------------------------------------------------------------------------|
| TC-LP-01 | (a) Pengguna membuka browser<br>(b) Pengguna mengakses URL aplikasi<br>(c) Sistem memuat halaman awal | Pengguna mengakses aplikasi tanpa melakukan autentikasi | –        | Halaman landing ditampilkan dengan informasi sistem serta navigasi menuju halaman login dan registrasi. |

## 2. Login

**Tabel 3.3** Skenario Pengujian Halaman Login

| ID       | Test Step                                                                                                                     | Skenario Pengujian                 | Data Uji                      | Hasil yang Diharapkan                                                     |
|----------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------|-------------------------------|---------------------------------------------------------------------------|
| TC-LG-01 | (a) Pengguna membuka halaman login<br>(b) Pengguna mengisi username dan kata sandi valid<br>(c) Pengguna menekan tombol login | Login dengan kredensial yang valid | username dan kata sandi valid | Sistem menerima kredensial dan mengarahkan pengguna ke halaman dashboard. |

| <b>ID</b> | <b>Test Step</b>                                                                                                                                                 | <b>Skenario Pengujian</b>                  | <b>Data Uji</b>                  | <b>Hasil yang Diharapkan</b>                                          |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|----------------------------------|-----------------------------------------------------------------------|
| TC-LG-02  | (a) Pengguna membuka halaman login<br>(b) Pengguna mengisi username valid<br>(c) Pengguna mengisi kata sandi salah<br>(d) Pengguna menekan tombol login          | Login dengan kata sandi yang salah         | username valid, kata sandi salah | Sistem menampilkan pesan kesalahan dan tetap berada di halaman login. |
| TC-LG-03  | (a) Pengguna membuka halaman login<br>(b) Pengguna mengisi username yang tidak terdaftar<br>(c) Pengguna mengisi kata sandi<br>(d) Pengguna menekan tombol login | Login dengan username yang tidak terdaftar | username tidak terdaftar         | Sistem menampilkan pesan bahwa akun tidak ditemukan.                  |
| TC-LG-04  | (a) Pengguna membuka halaman login<br>(b) Pengguna tidak mengisi username atau kata sandi<br>(c) Pengguna menekan tombol login                                   | Login dengan field kosong                  | username atau kata sandi kosong  | Sistem menampilkan validasi bahwa seluruh field wajib diisi.          |

| ID       | Test Step                                                                                                                   | Skenario Pengujian                     | Data Uji         | Hasil yang Diharapkan                                                     |
|----------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------|------------------|---------------------------------------------------------------------------|
| TC-LG-05 | (a) Pengguna membuka halaman login<br>(b) Pengguna mengisi ulang kredensial yang benar<br>(c) Pengguna menekan tombol login | Login berhasil setelah percobaan gagal | Kredensial valid | Sistem menerima kredensial dan mengarahkan pengguna ke halaman dashboard. |

### 3. Register

**Tabel 3.4** Skenario Pengujian Halaman Register

| ID       | Test Step                                                                                                                                      | Skenario Pengujian                   | Data Uji                              | Hasil yang Diharapkan                                                        |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|---------------------------------------|------------------------------------------------------------------------------|
| TC-RG-01 | (a) Pengguna membuka halaman register<br>(b) Pengguna mengisi seluruh field registrasi dengan data valid<br>(c) Pengguna menekan tombol daftar | Registrasi dengan data yang valid    | Data registrasi lengkap dan valid     | Sistem menyimpan data pengguna dan akun dapat digunakan untuk login.         |
| TC-RG-02 | (a) Pengguna membuka halaman register<br>(b) Pengguna tidak mengisi salah satu atau beberapa field<br>(c) Pengguna menekan tombol daftar       | Registrasi dengan data tidak lengkap | Salah satu atau beberapa field kosong | Sistem menampilkan pesan validasi bahwa seluruh data registrasi wajib diisi. |

| ID       | Test Step                                                                                                                              | Skenario Pengujian                               | Data Uji                 | Hasil yang Diharapkan                                |
|----------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|--------------------------|------------------------------------------------------|
| TC-RG-03 | (a) Pengguna membuka halaman register<br>(b) Pengguna mengisi username yang sudah terdaftar<br>(c) Pengguna menekan tombol daftar      | Registrasi dengan username yang sudah terdaftar  | username sudah terdaftar | Sistem menampilkan pesan bahwa akun sudah terdaftar. |
| TC-RG-04 | (a) Pengguna membuka halaman register<br>(b) Pengguna mengisi ulang data registrasi dengan benar<br>(c) Pengguna menekan tombol daftar | Registrasi berhasil setelah kesalahan sebelumnya | Data registrasi valid    | Sistem menerima data dan memproses registrasi akun.  |

#### 4. Dashboard

**Tabel 3.5** Skenario Pengujian Halaman Dashboard

| ID       | Test Step                                                                           | Skenario Pengujian              | Data Uji                | Hasil yang Diharapkan                                                   |
|----------|-------------------------------------------------------------------------------------|---------------------------------|-------------------------|-------------------------------------------------------------------------|
| TC-DB-01 | (a) Pengguna berhasil login<br>(b) Sistem mengarahkan pengguna ke halaman dashboard | Membuka dashboard setelah login | Data dashboard tersedia | Sistem menampilkan ringkasan data dan visualisasi utama pada dashboard. |



| <b>ID</b> | <b>Test Step</b>                                                                                                   | <b>Skenario Pengujian</b>                    | <b>Data Uji</b>          | <b>Hasil yang Diharapkan</b>                                                |
|-----------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------|--------------------------|-----------------------------------------------------------------------------|
| TC-DB-02  | (a) Pengguna login ke sistem<br>(b) Sistem memuat halaman dashboard<br>(c) Data dashboard belum tersedia           | Membuka dashboard dengan data belum tersedia | Data belum tersedia      | Sistem menampilkan indikator pemuatan atau pesan informasi kepada pengguna. |
| TC-DB-03  | (a) Pengguna membuka dashboard<br>(b) Pengguna berpindah ke menu lain<br>(c) Pengguna kembali ke halaman dashboard | Navigasi keluar dan kembali ke dashboard     | –                        | Sistem menampilkan data dashboard secara konsisten tanpa kehilangan data.   |
| TC-DB-04  | (a) Pengguna berada di halaman dashboard<br>(b) Sistem menerima pembaruan data dari server                         | Pembaruan data dashboard dari server         | Data diperbarui          | Sistem memperbarui tampilan dashboard sesuai dengan data terbaru.           |
| TC-DB-05  | (a) Pengguna membuka halaman dashboard<br>(b) Beberapa komponen memuat data dari sumber yang sama                  | Konsistensi data antar komponen dashboard    | Data yang sama digunakan | Sistem menampilkan data yang konsisten pada seluruh komponen dashboard.     |

## 5. Sentiment

**Tabel 3.6** Skenario Pengujian Halaman Dashboard Sentiment

| ID       | Test Step                                                                                                                         | Skenario Pengujian                                     | Data Uji                     | Hasil yang Diharapkan                                                       |
|----------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|------------------------------|-----------------------------------------------------------------------------|
| TC-ST-01 | (a) Pengguna berhasil login<br>(b) Pengguna membuka halaman dashboard sentiment                                                   | Membuka halaman dashboard sentiment                    | Data sentimen tersedia       | Sistem menampilkan visualisasi dan ringkasan hasil analisis sentimen.       |
| TC-ST-02 | (a) Pengguna membuka halaman dashboard sentiment<br>(b) Data sentimen belum tersedia                                              | Membuka dashboard sentiment dengan data belum tersedia | Data sentimen belum tersedia | Sistem menampilkan indikator pemuatan atau pesan informasi kepada pengguna. |
| TC-ST-03 | (a) Pengguna berada di halaman dashboard sentiment<br>(b) Sistem menerima pembaruan data sentimen dari server                     | Pembaruan data sentimen dari server                    | Data sentimen diperbarui     | Sistem memperbarui visualisasi sesuai dengan data sentimen terbaru.         |
| TC-ST-04 | (a) Pengguna membuka dashboard sentiment<br>(b) Pengguna berpindah ke halaman lain<br>(c) Pengguna kembali ke dashboard sentiment | Navigasi keluar dan kembali ke dashboard sentiment     | –                            | Sistem menampilkan data sentimen secara konsisten tanpa kehilangan data.    |

| <b>ID</b> | <b>Test Step</b>                                                                                 | <b>Skenario Pengujian</b>                | <b>Data Uji</b>                   | <b>Hasil yang Diharapkan</b>                                              |
|-----------|--------------------------------------------------------------------------------------------------|------------------------------------------|-----------------------------------|---------------------------------------------------------------------------|
| TC-ST-05  | (a) Pengguna membuka dashboard sentiment<br>(b) Beberapa komponen memuat data sentimen yang sama | Konsistensi data sentimen antar komponen | Data sentimen yang sama digunakan | Sistem menampilkan data yang konsisten pada seluruh komponen visualisasi. |

## 6. Recommendation

**Tabel 3.7** Skenario Pengujian Halaman Dashboard Rekomendasi Konten

| <b>ID</b> | <b>Fitur</b>          | <b>Skenario Pengujian</b>                                       | <b>Data Uji</b>                 | <b>Hasil yang Diharapkan</b>                                              |
|-----------|-----------------------|-----------------------------------------------------------------|---------------------------------|---------------------------------------------------------------------------|
| TC-RC-01  | Dashboard Rekomendasi | Pengguna membuka halaman dashboard rekomendasi konten           | Data rekomendasi tersedia       | Sistem menampilkan daftar rekomendasi berdasarkan hasil analisis sentimen |
| TC-RC-02  | Dashboard Rekomendasi | Pengguna membuka halaman rekomendasi dengan data belum tersedia | Data rekomendasi belum tersedia | Sistem menampilkan indikator pemuatan atau pesan informasi                |

| <b>ID</b> | <b>Fitur</b>          | <b>Skenario Pengujian</b>                                     | <b>Data Uji</b>               | <b>Hasil yang Diharapkan</b>                                |
|-----------|-----------------------|---------------------------------------------------------------|-------------------------------|-------------------------------------------------------------|
| TC-RC-03  | Dashboard Rekomendasi | Pembaruan data rekomendasi dari server                        | Data rekomendasi diperbarui   | Sistem memperbarui tampilan rekomendasi sesuai data terbaru |
| TC-RC-04  | Dashboard Rekomendasi | Navigasi ke halaman lain dan kembali ke dashboard rekomendasi | –                             | Sistem menampilkan data rekomendasi secara konsisten        |
| TC-RC-05  | Dashboard Rekomendasi | Konsistensi rekomendasi dengan data sentimen                  | Data sentimen terkait berubah | Rekomendasi konten menyesuaikan perubahan data sentimen     |

## 7. Scraper

**Tabel 3.8** Skenario Pengujian Halaman Data Scraper

| <b>ID</b> | <b>Test Step</b>                                                                   | <b>Skenario Pengujian</b>    | <b>Data Uji</b>              | <b>Hasil yang Diharapkan</b>                                              |
|-----------|------------------------------------------------------------------------------------|------------------------------|------------------------------|---------------------------------------------------------------------------|
| TC-SC-01  | (a) Pengguna membuka halaman data scraper<br>(b) Sistem memuat data hasil scraping | Membuka halaman data scraper | Data hasil scraping tersedia | Sistem menampilkan daftar data hasil scraping yang diperoleh dari server. |

| <b>ID</b> | <b>Test Step</b>                                                                                                                    | <b>Skenario Pengujian</b>                           | <b>Data Uji</b>          | <b>Hasil yang Diharapkan</b>                                                |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|--------------------------|-----------------------------------------------------------------------------|
| TC-SC-02  | (a) Pengguna membuka halaman data scraper<br>(b) Data hasil scraping belum tersedia                                                 | Membuka halaman scraper dengan data belum tersedia  | Data belum tersedia      | Sistem menampilkan indikator pemuatan atau pesan informasi kepada pengguna. |
| TC-SC-03  | (a) Pengguna membuka halaman data scraper<br>(b) Pengguna memilih salah satu dataset                                                | Pemilihan data untuk dianalisis                     | Dataset tertentu dipilih | Sistem menandai data terpilih untuk digunakan pada proses analisis.         |
| TC-SC-04  | (a) Pengguna membuka halaman data scraper<br>(b) Pengguna berpindah ke halaman lain<br>(c) Pengguna kembali ke halaman data scraper | Navigasi keluar dan kembali ke halaman data scraper | –                        | Sistem menampilkan data scraper secara konsisten tanpa kehilangan data.     |

## 8. Chatbot

**Tabel 3.9** Skenario Pengujian Chatbot

| ID       | Test Step                                                                                                                                 | Skenario Pengujian                            | Data Uji                 | Hasil yang Diharapkan                                                       |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|--------------------------|-----------------------------------------------------------------------------|
| TC-CB-01 | (a) Pengguna berhasil login<br>(b) Pengguna berada di halaman dashboard<br>(c) Pengguna membuka panel chatbot yang bersifat <i>sticky</i> | Membuka antarmuka chatbot pada dashboard      | –                        | Sistem menampilkan antarmuka chatbot dan siap menerima input pengguna.      |
| TC-CB-02 | (a) Pengguna membuka chatbot pada dashboard<br>(b) Pengguna memasukkan pertanyaan teks<br>(c) Pengguna mengirimkan pertanyaan             | Mengirimkan pertanyaan melalui chatbot        | Pertanyaan teks valid    | Sistem menampilkan respons chatbot sesuai dengan pertanyaan yang diberikan. |
| TC-CB-03 | (a) Pengguna membuka chatbot<br>(b) Pengguna mengirimkan input kosong                                                                     | Mengirimkan input kosong pada chatbot         | Input kosong             | Sistem menampilkan pesan validasi atau informasi bahwa input tidak valid.   |
| TC-CB-04 | (a) Pengguna membuka chatbot<br>(b) Pengguna mengirimkan pertanyaan di luar konteks sistem                                                | Mengirimkan pertanyaan di luar konteks sistem | Pertanyaan tidak relevan | Sistem menampilkan respons atau pesan informasi yang sesuai.                |

| <b>ID</b> | <b>Test Step</b>                                                                         | <b>Skenario Pengujian</b>         | <b>Data Uji</b>               | <b>Hasil yang Diharapkan</b>                                                                                                                                       |
|-----------|------------------------------------------------------------------------------------------|-----------------------------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TC-CB-05  | (a) Pengguna membuka chatbot<br>(b) Pengguna melakukan beberapa interaksi berturut-turut | Interaksi berulang dengan chatbot | Beberapa pertanyaan berurutan | Sistem memproses interaksi chatbot secara berurutan dengan menonaktifkan input selama proses berlangsung dan mengaktifkannya kembali setelah respons ditampilkan.. |

## 9. Logout

**Tabel 3.10** Skenario Pengujian Logout

| <b>ID</b> | <b>Test Step</b>                                                              | <b>Skenario Pengujian</b>    | <b>Data Uji</b> | <b>Hasil yang Diharapkan</b>                                      |
|-----------|-------------------------------------------------------------------------------|------------------------------|-----------------|-------------------------------------------------------------------|
| TC-LO-01  | (a) Pengguna berada dalam kondisi login<br>(b) Pengguna menekan tombol logout | Melakukan logout dari sistem | –               | Sistem mengakhiri sesi pengguna dan mengarahkan ke halaman login. |

| <b>ID</b> | <b>Test Step</b>                                                                        | <b>Skenario Pengujian</b>             | <b>Data Uji</b> | <b>Hasil yang Diharapkan</b>                                    |
|-----------|-----------------------------------------------------------------------------------------|---------------------------------------|-----------------|-----------------------------------------------------------------|
| TC-LO-02  | (a) Pengguna telah melakukan logout<br>(b) Pengguna mencoba mengakses halaman dashboard | Mengakses dashboard setelah logout    | –               | Sistem menolak akses dan mengarahkan pengguna ke halaman login. |
| TC-LO-03  | (a) Pengguna menutup sesi aplikasi<br>(b) Pengguna membuka kembali aplikasi             | Membuka ulang aplikasi setelah logout | –               | Sistem meminta pengguna untuk melakukan login kembali.          |



## DAFTAR PUSTAKA

- Alviani, N. A., Studi, P., Fakultas, M., and Bangsa, U. B. (2025). Transformasi Digital pada UMKM dalam Meningkatkan Daya Saing Pasar. *Master Manajemen*, 3(1):134–140.
- Aniley, D., Alemneh, E., and Abeba, G. (2024). Selection of software development life cycle models using machine learning approach. *International Journal of Computer Applications*, 186:975–8887.
- Evergreen, P. (2023). Selecting a State Management Strategy for Modern Web Frontend Applications. *Tampere University*, page 66.
- Fajarini, S., Kurniawati, J., and Yuliani, F. (2025). Social media sentiment analysis as a new tool for predicting market trends and consumer behaviour. *Proceeding of International Conference on Social Science and Humanity*, 2:899–909.
- Highlights, F. (2025). TanStack Query: A Powerful Tool for Data Management in React. Diakses pada 07 Jan 2026.
- Islam, M. M. (2025). The impact of data-driven web frameworks on performance and scalability of u.s. enterprise applications. *International Journal of Business and Economics Insights*, 05:523–558.
- Joseph, T. (2024). Natural Language Processing (NLP) for Sentiment Analysis in Social Media. *International Journal of Computing and Engineering*, 6(2):35–48.
- Luz, H. (2025). Comparing performance of redux, mobx, and react query. ResearchGate. Preprint, ResearchGate.
- Maharani, L. (2024). Analisis Pola Dan Tren Penggunaan Data Media Sosial Dengan Teknik Data Mining. *Logicloom*, 1(2):1–21.
- Mardhatilah, D., Omar, A., and Septiari, E. D. (2024). BUILDING CONSUMER ENGAGEMENT IN SOCIAL MEDIA: A SYSTEMATIC LITERATURE REVIEW. *JOURNAL OF BUSINESS MANAGEMENT AND ACCOUNTING*, 14(1):1–35.
- Maulida, M., Zahro, F., Hakim, R., and Akbar, M. S. (2025). PT. Media Akademik Publisher PENGUJIAN BLACK BOX TESTING PADA SISTEM WEBSITE PEMESANAN ONLINE TOKO AYAM KRISPY. *Jurnal Media Akademik (Jma)*, 3(5):3031–5220.
- Micheal, D. (2025). React Query and Lazy Loading: Performance Optimization Best Practices. Preprint, ResearchGate.

- Owusu, K. (2025). REST API Architecture in React A Production Guide.
- Purnama, I., Setiani, Y., Ari, F., and Wibisono, N. (2025). Analisis dan Visualisasi Data Menggunakan Looker Studio Pada Dataset New York City Property Sales. *Jurnal Minfo Polgan*, 13:2222–2234.
- Putra, F. P. E., Efendi, R. W., Tamam, A. B., and Pramadi, W. A. (2025). Trends and best practices in api-based web development using laravel and react. *Brilliance: Journal of Information and Software Engineering*, 5(1):1–10.
- Rahman, A. and Prihanto, A. (2024). Optimisasi Kinerja Aplikasi Fitness Berbasis Next.js Melalui Penerapan Metode Caching Pada PT. Anugerah Wijaya Raga. *Journal of Informatics and Computer Science (JINACS)*, 6(02):333–340.
- Rathore, S., Nawkhare, R., Sharma, N., Chaudhary, N., Chakole, S., and Vishwakrama, B. (2025). Effective data visualization techniques for business decision-makers. *International Research Journal on Advanced Engineering and Management (IRJAEM)*, 3:2029–2036.
- React (2026). React: A javascript library for building user interfaces. <https://react.dev/>. Diakses pada 07 Jan 2026.
- Sastra, R. and Sutawinata, A. M. (2023). Perancangan Aplikasi SIP-PTK Sekolah Dasar Negeri Guntur 01 Menggunakan Model Fountain. *INSANtek*, 4(2):63–68.
- Siva, F., Assegaf, S. M. U., Pahlevi, S. A., and Yaqin, M. A. (2023). Survey metode-metode software development life cycle dengan metode systematic literature review. *Lembaga Penelitian dan Pengabdian Masyarakat*, 5(2).
- Sofyan, S. and Agusman (2025). ANALISIS KESIAPAN UMKM MENGHADAPI EKONOMI DIGITAL. *Smart Jurnal Ilmiah*, IX(1):1–4.
- Suryani, A. and Komputer, I. (2024). ANALISIS SENTIMEN PENGGUNA BERBASIS DATA. *Logicloom.id*, 1(4):1–21.
- Tanstack LCC (2025). Tanstack Query Official Documentation. Diakses pada 07 Jan 2026.
- Trulline, P. (2021). Pemasaran produk UMKM melalui media sosial dan e-commerce. *Jurnal Manajemen Komunikasi*, 5(2):259.